

SPIKING NEURAL NETWORKS

Michele Rossi

michele.rossi@unipd.it

Dept. of Information Engineering

Dept. of Mathematics



**UNIVERSITÀ
DEGLI STUDI
DI PADOVA**

Outline

- Introduction & motivation
- Neuron model [SNN-Book-2019]
 - Leaky Integrate and Fire (LIF) model
 - Discrete time neuron model
- Neural network model [SPM-2019, ProcIEEE-2023]
 - Discrete RNN model
- Supervised training [SNN-SPM-2019]
 - Surrogate gradient descent
- Deep spiking neural networks [NeurIPS-2021]
- Conclusions



Introduction & motivation

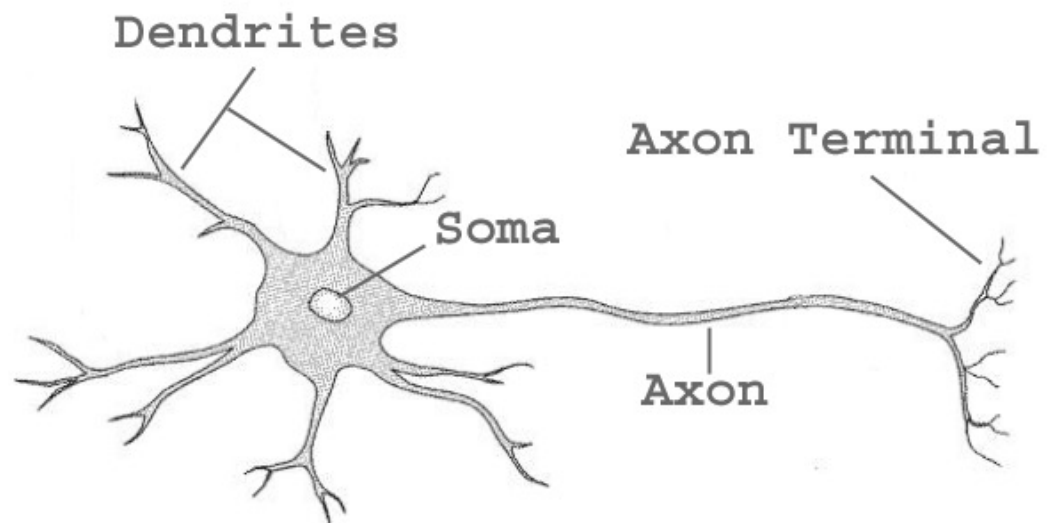
- Neural networks **are getting huge** (billions of pars) and their training is often **highly energy demanding**
- State-of-the-art solutions (e.g., transformer based) rely on convolutions and “correlations” to capture the structure and the relations underpinning data
- Current optimization approaches involve
 - **Optimizations:** weight quantization, pruning, etc.
 - **Architectural designs:** reservoir computing, extreme learning machines, etc.
- Such approaches **do not address the core of the problem:** current architectures inherently perform **dense processing** of data in both time and space

Some approaches to improve efficiency of deep learning models

- New designs + optimizations (quantization, pruning, etc.)
- Continual learning (freshness via model update)
- Unsupervised or semi-supervised learning (data efficiency)
- Reduce data transfer & communication
- Use of analog & mixed signal circuitry
- Sparsity of activation & connections

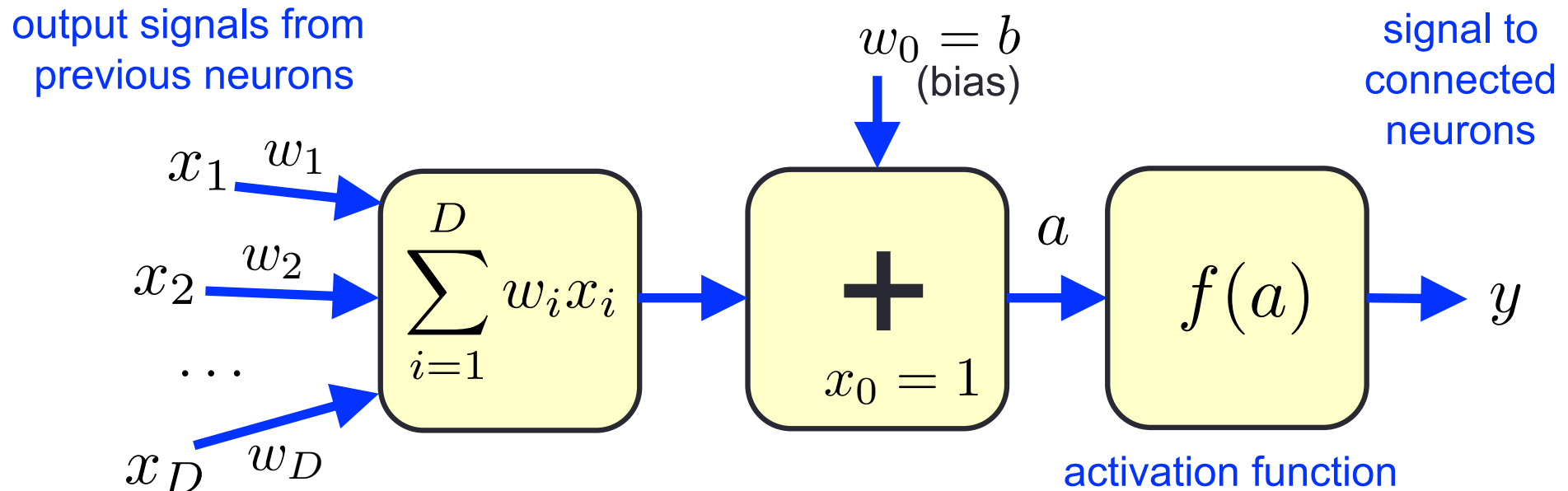
Biological neurons

- Neuron
 - basic elements in the brain
 - **soma**: “central processing unit” performs a **nonlinear** processing step. If the total input arriving at the soma exceeds a threshold → an output signal is generated (a so called *spike*)



2nd gen neural networks

- Approximate the biological neuron by assuming that input and output signals are real numbers

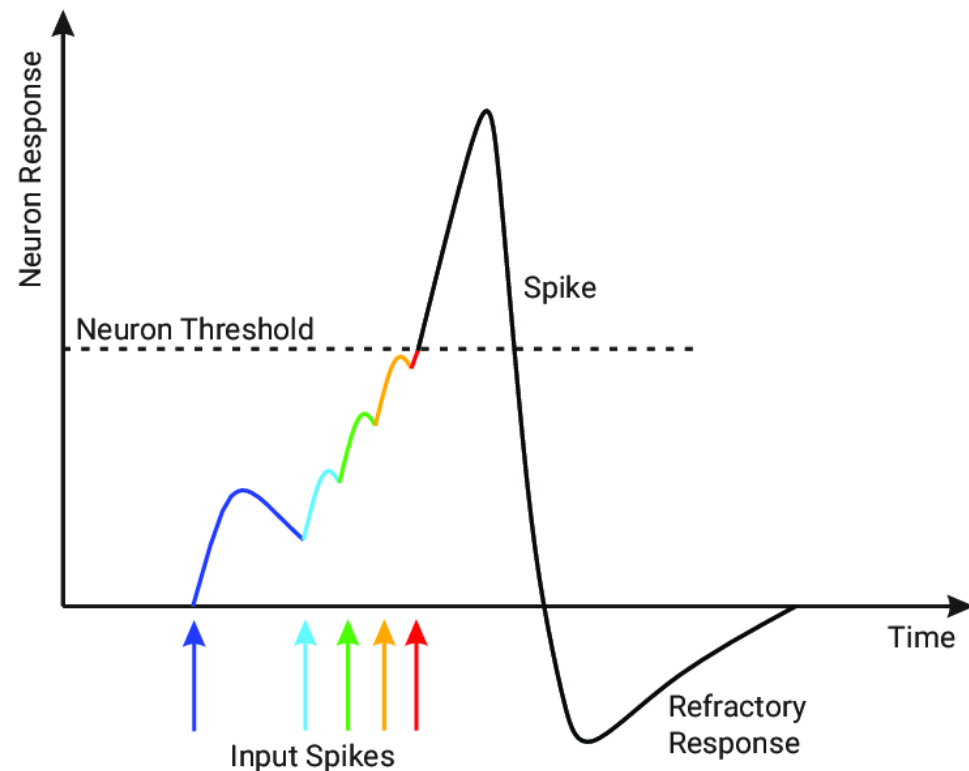


$$y = f(a) = f \left(\sum_{j=1}^D w_j x_j + b \right) = f \left(\sum_{j=0}^D w_j x_j \right)$$

Biological neurons: spikes

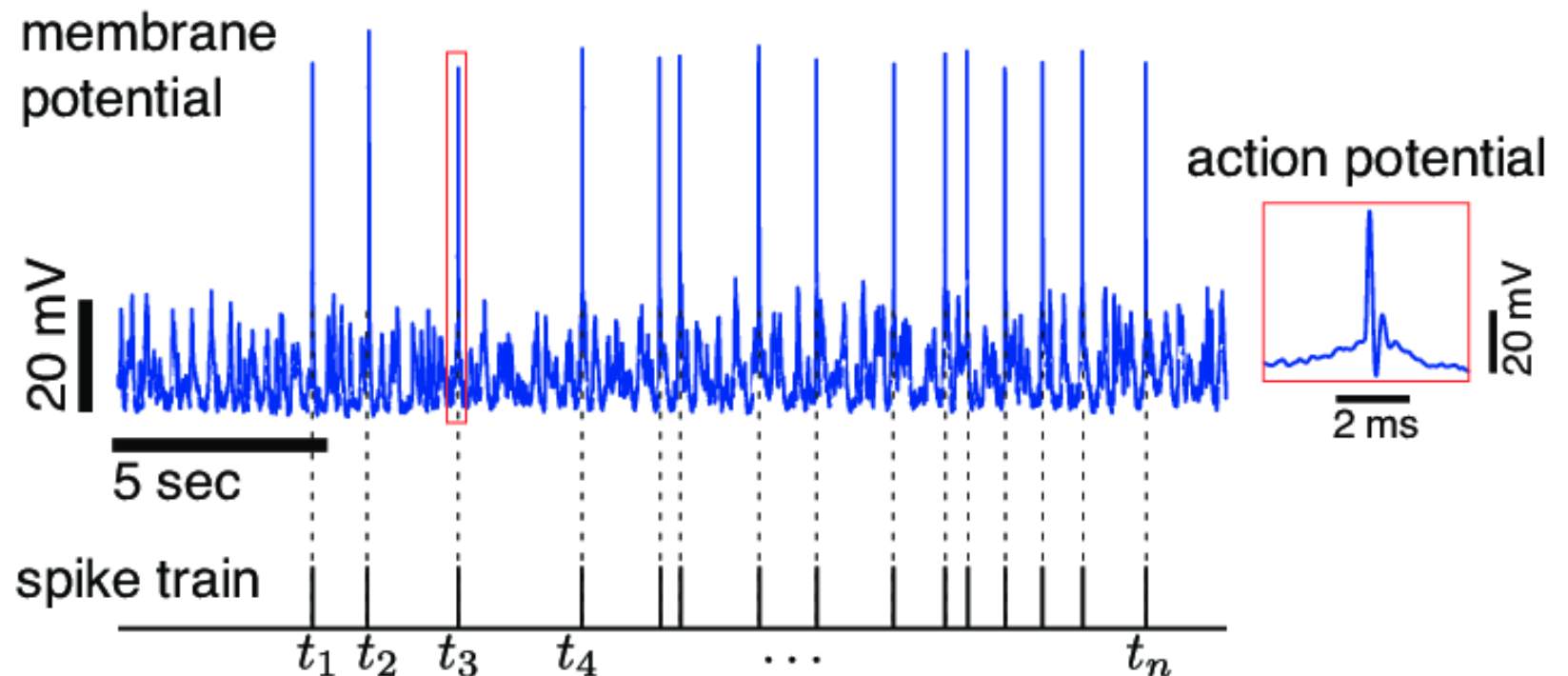
- Spikes

- Neuron output signal consists of *spikes* → short pulses (1-2 ms) with a typical amplitude of 100 mV. Their form *does not change* as they propagate along the axon



Spike trains

- Spikes are generated when the potential of a neuron membrane **exceeds a threshold**. Spikes in a train are well separated (there is a *refractory* period after the emission of a spike during which further spikes **are inhibited**)



Membrane potential $u(t)$

- Is the potential difference between the interior of the neuronal cell and its surroundings
- Without any input, the neuron is **at rest** and has a (negative) **rest membrane potential** u_{rest} (about -65 mV)
- After the arrival of a spike, the potential changes
 - **Positive change:** excitatory synapse
 - **Negative change:** inhibitory synapse
- Our interest
 - **Time evolution of the membrane potential** $u_i(t)$ of neuron i
 - In resting conditions, it holds $u_i(t) = u_{\text{rest}}$
 - If presynaptic neuron j fires a spike at **time** $t=0$, we get a response

$$\epsilon_{ij}(t) := u_i(t) - u_{\text{rest}}$$

MODELING NEURON DYNAMICS

The Leaky Integrate and Fire (LIF) neuron model

Leaky Integrate and Fire (LIF) model

From [Gerstner-2014]

<<The model makes use of the fact that neuronal action potentials of a given neuron always have roughly the same form. If the shape of an action potential is always the same, then the shape cannot be used to transmit information: rather information is contained in the presence or absence of a spike. Therefore, action potentials are reduced to “events” that happen at a precise moment in time>>

[Gerstner-2014] Wulfram Gerstner, Werner M. Kistler, Richard Naud and Liam Paninski, “Neuronal Dynamics: from single neurons to networks and models of cognition,” *Cambridge University Press*, 2014.

Leaky Integrate and Fire (LIF) Model

Objective: track the action potential $u(t)$ of a neuron

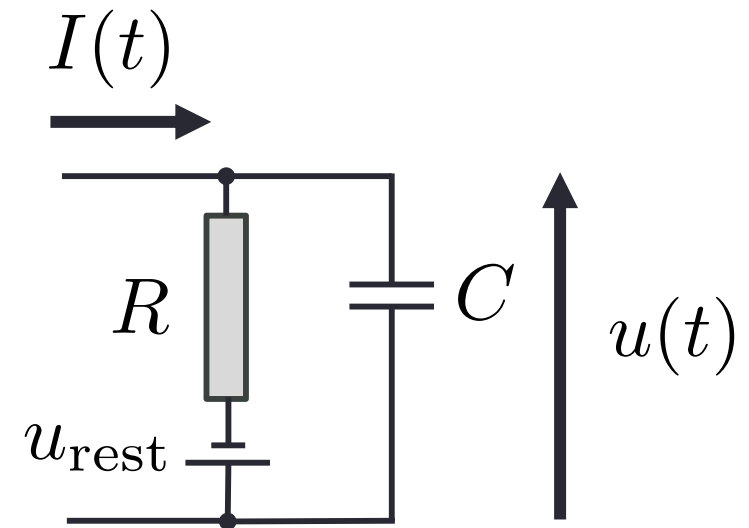
The following **RC circuit** can be used to describe the neuron behaviour

- **R**: input resistance
- **C**: neuron membrane capacitor

$I(t)$ input current (spikes)

u_{rest} membrane potential at rest

$u(t)$ membrane potential at time t



Leaky Integrate and Fire (LIF) Model

$$I(t) = I_R(t) + I_C(t)$$

$$I(t) = \frac{u(t) - u_{\text{rest}}}{R} + C \frac{du(t)}{dt}$$

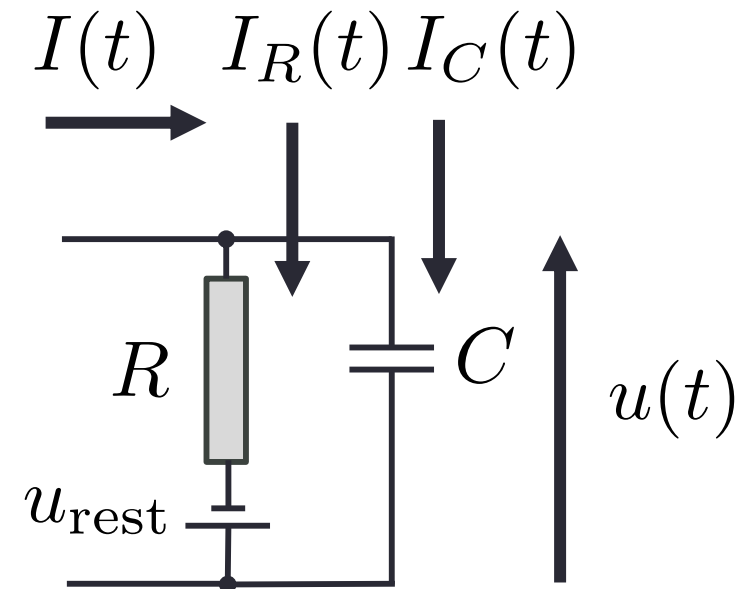
current on
resistor R

current on
capacitor C

$I(t)$ input (injected) current (spikes)

u_{rest} membrane potential at rest

$u(t)$ membrane potential at time t



Leaky Integrate and Fire (LIF) Model

$$\tau_m \frac{du(t)}{dt} = -[u(t) - u_{\text{rest}}] + RI(t) \quad (1) \quad \tau_m := RC \begin{array}{l} \text{membrane} \\ \text{time constant} \end{array}$$

- **Mathematician:** this is a linear differential equation
- **Engineer:** the equation of a “leaky integrator”
- **Neuroscientist:** equation of a passive neuron membrane

Assuming that for a certain time t_0 , the injected current is $I(t)=0$ for $t \geq t_0$, and the membrane potential is $u(t_0) = u_{\text{rest}} + \Delta u$, with $\Delta u > 0$, the equation admits a closed form solution

Case 1: LIF discharge, input current $I_0=0$

$$\tau_m \frac{du(t)}{dt} = -[u(t) - u_{\text{rest}}], \quad t \geq t_0 \quad \text{with: } I(t) = 0$$

$$u(t) - u_{\text{rest}} = \Delta u \exp\left(-\frac{t - t_0}{\tau_m}\right), \quad \text{for } t > t_0 \quad (2)$$

$$\text{with: } u(t_0) = u_{\text{rest}} + \Delta u$$

In the absence of input, $u(t)$ decays exponentially (discharging capacitor) to its resting value. The membrane time constant $\tau_m=RC$ is the *characteristic time of the decay*. For a typical neuron it is in the range of 10ms, which is longer than the duration of a spike (on the order of 1ms)

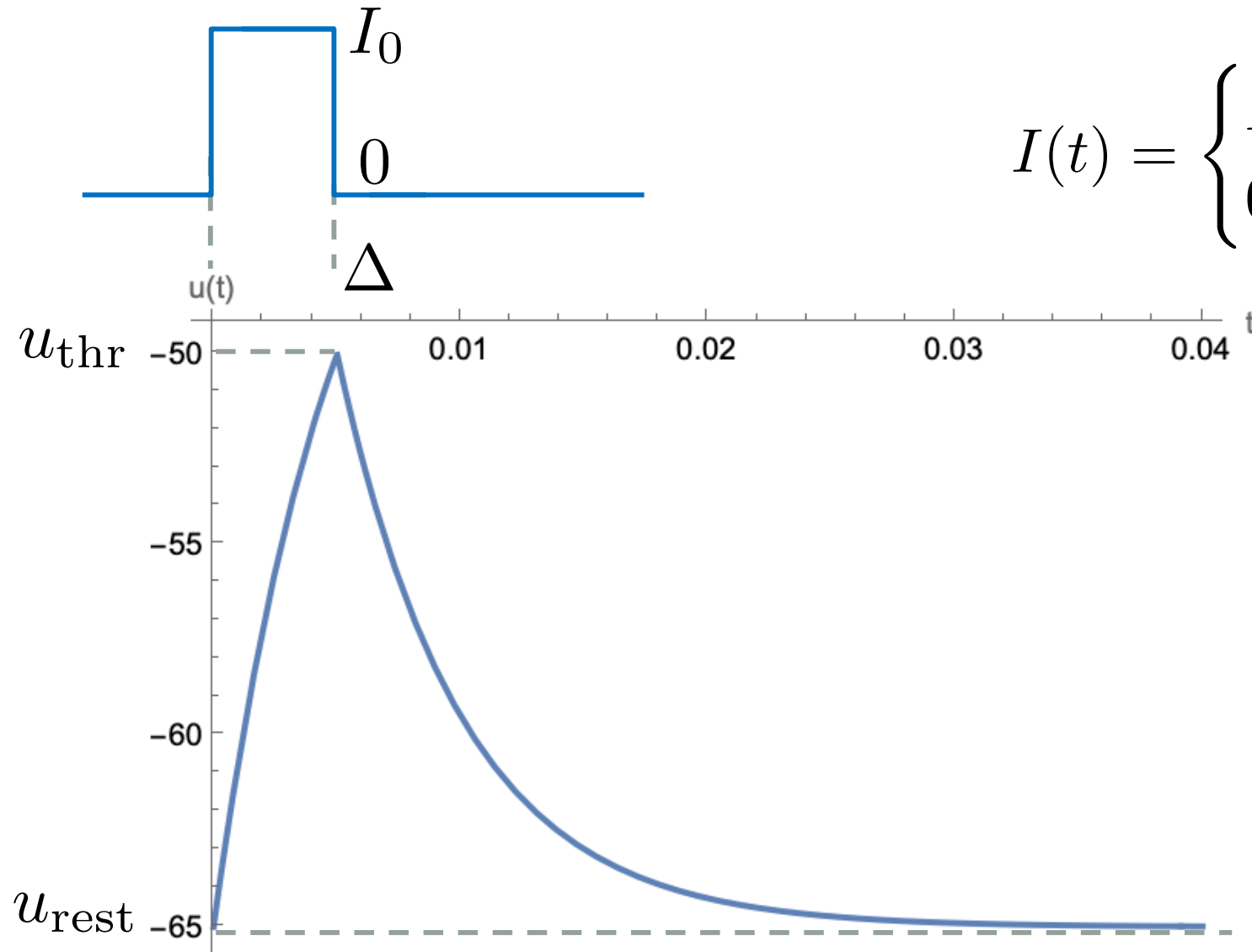
Case 2: LIF charge with constant current I_0

- At $t_0=0$ a constant current I_0 is generated, starting at t_0 and ending at $t=\Delta$
- For the sake of simplicity, we assume that $u(0)=u_{\text{rest}}$
- $u(t)$ in this case is found by integrating Eq. (1) with the initial condition $u(0)=u_{\text{rest}}$
- The solution in $0 < t < \Delta$ is – **loading capacitor**:

$$u(t) = u_{\text{rest}} + RI_0 \left[1 - \exp \left(-\frac{t}{t_0} \right) \right] \quad (3)$$

- After $t \geq \Delta$, the previous equation (2) applies with initial charge $u(\Delta)$ (computed using Eq. (3))

Pulse input (constant current I_0)



$$I(t) = \begin{cases} I_0 & t \in [0, \Delta] \\ 0 & \text{otherwise} \end{cases}$$

$$\Delta = 5 \text{ ms}$$

$$C = 100 \text{ pF}$$

$$R = 50 \text{ M}\Omega$$

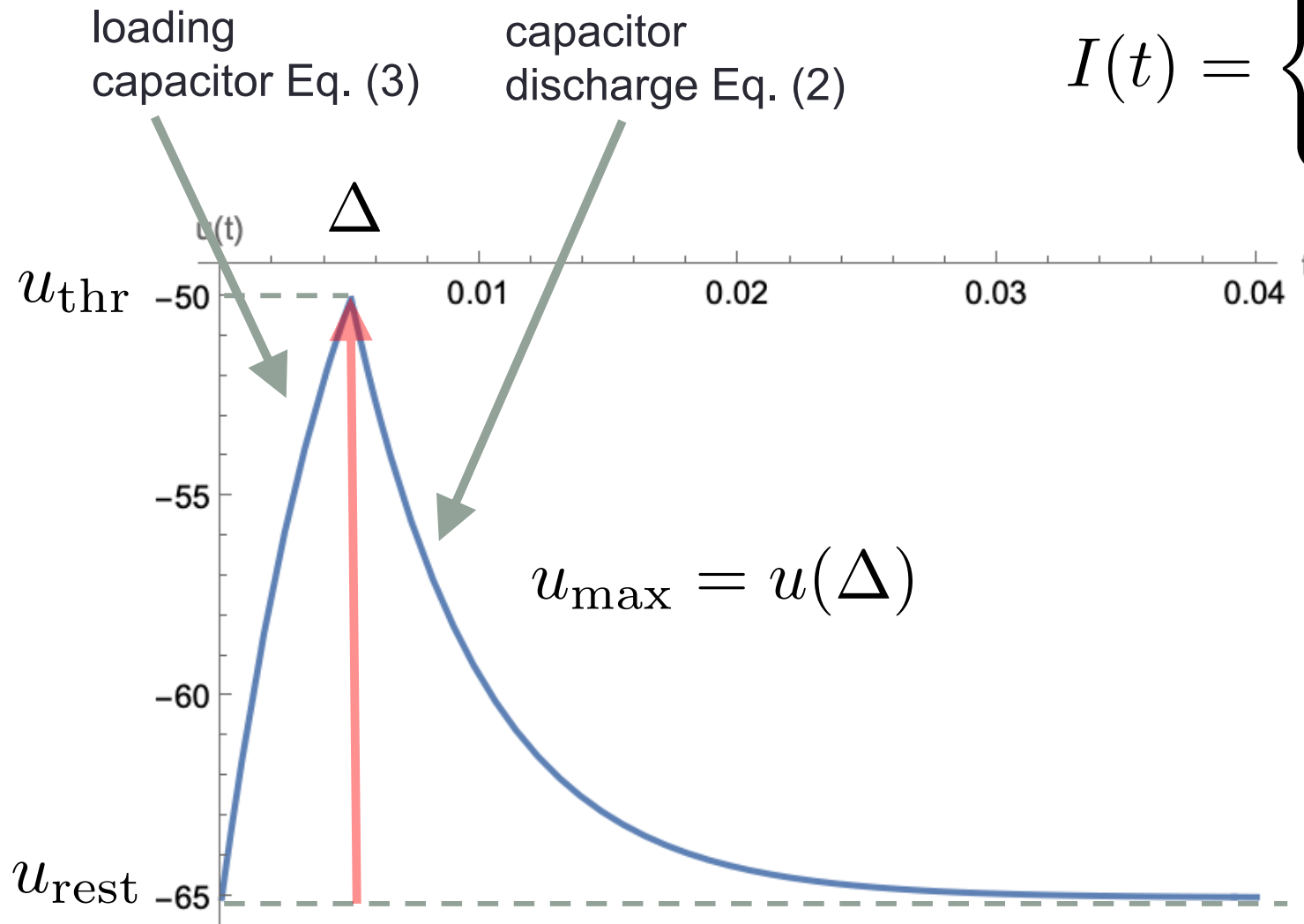
$$u_{\text{rest}} = -70 \text{ mV}$$

$$u_{\text{thr}} = -50 \text{ mV}$$

$$I_0 \simeq 0.475 \mu\text{A}$$

Pulse input (constant current I_0)

$$I(t) = \begin{cases} I_0 & t \in [0, \Delta] \\ 0 & \text{elsewhere} \end{cases}$$



$$\Delta = 5 \text{ ms}$$

$$C = 100 \text{ pF}$$

$$R = 50 \text{ M}\Omega$$

$$u_{rest} = -70 \text{ mV}$$

$$u_{thr} = -50 \text{ mV}$$

$$I_0 \simeq 0.475 \mu\text{A}$$

Short pulses

- For short pulses of duration $\Delta \ll \tau_m$, using Eq. (3)

$$u(\Delta) = u_{\text{rest}} + RI_0 \left[1 - \exp \left(-\frac{\Delta}{\tau_m} \right) \right] \simeq u_{\text{rest}} + RI_0 \frac{\Delta}{\tau_m} \quad (3\text{bis})$$

due to Taylor expansion of exp()

- We now make the current pulses (of duration Δ) shorter and shorter by maintaining the integral of the current unchanged and equal to the total charge q

$$\int_0^{\Delta} I(t) dt = I_0 \Delta = q$$

- It holds $I_0 = q/\Delta$ and, plugging into (3bis):

$$u(\Delta) = u_{\text{rest}} + Rq/\tau_m$$

Short pulses

- Imposing $\int_0^{\Delta} I(t)dt = I_0\Delta = q$
- As Δ gets shorter, in the limit, the final voltage $u(\Delta)$ of the cell membrane depends on the charge q and not on Δ
- Thus, we can set $I(t) = q\delta(t)$
- The **amount of charge q** transferred to the neuron remains constant, and **the spike is modeled by an impulse**, so Eq. (1) becomes (4)

$$\tau_m \frac{du(t)}{dt} = -[u(t) - u_{\text{rest}}] + Rq\delta(t) \quad (4)$$

Effect of an input Spike at time $t_0=0$

- In the limit, the input current is modelled as a Dirac delta

$$\tau_m \frac{du(t)}{dt} = -[u(t) - u_{\text{rest}}] + Rq\delta(t) \quad (4)$$

The solution of this differential eq. is:

$$u(t) = \begin{cases} u_{\text{rest}} & t \leq 0 \\ u_{\text{rest}} + q \frac{R}{\tau_m} \exp\left(-\frac{t}{\tau_m}\right) & t > 0 \end{cases}$$

Green's function of the linear differential equation

The input spike at time $t_0=0$ is modelled using a Dirac delta

Input spikes

- As input spikes arrive at a neuron
 - The membrane potential $u(t)$ increases (accumulates)
- However, $u(t)$ cannot increase indefinitely large
 - There is a point at which the neuron itself fires
 - Releasing the accumulated “energy” and emitting an **output spike**
 - This point depends on a **neuron specific threshold**

Firing times

- The **firing time** $t^{(f)}$, $f = 1, 2, 3, \dots$ refers to the time instant when a neuron emits an action potential (an “output spike”). The firing time is defined by a threshold criterion

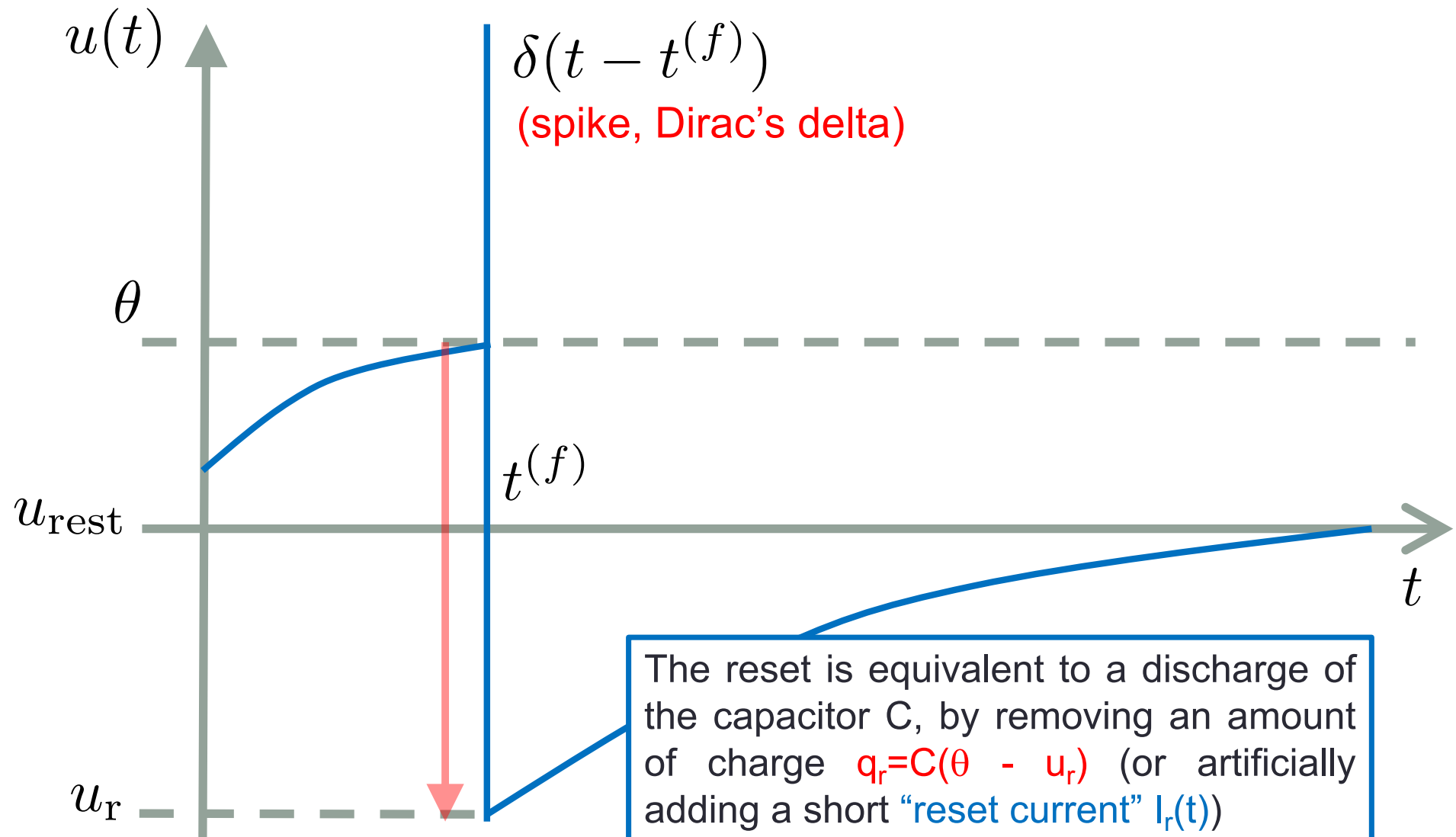
$$t^{(f)} : u(t^{(f)}) = \theta \quad (5)$$

- Immediately after t^f the potential is reset to a value

$$\lim_{\epsilon \rightarrow 0^+} u(t^{(f)} + \epsilon) = u_r$$

- From this point on the dynamics of $u(t)$ follows again Eq. (1) until the next threshold crossing occurs

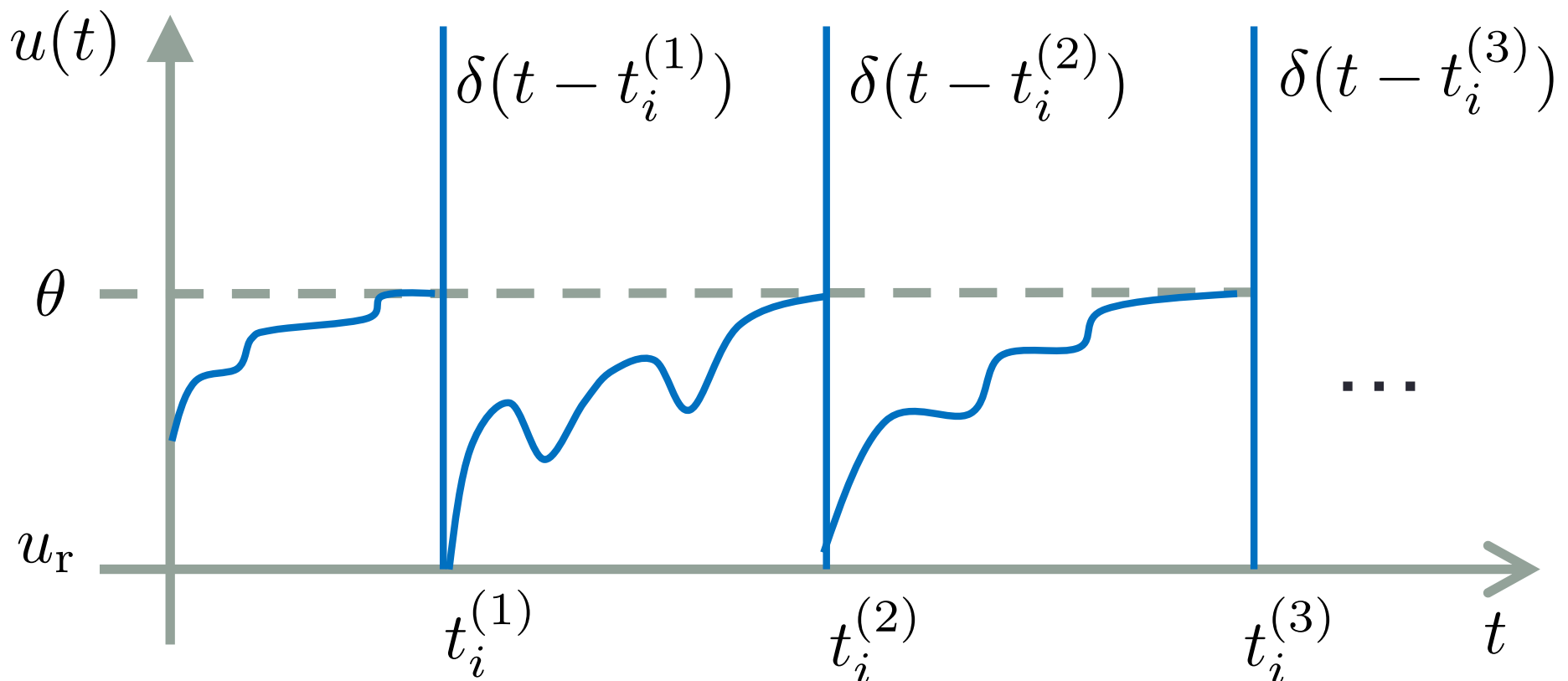
Mathematical model of a LIF neuron



Firing times

$$S_i(t) = \sum_f \delta(t - t_i^{(f)}) \quad (6)$$

- We denote the spike train $S_i(t)$ from neuron i as the sequence of firing times $t^{(f)} = \{t \mid u(t) = \theta\}$



Reset current $I_r(t)$

- Each time $u(t)$ reaches the threshold θ , the membrane potential $u(t)$ is reset to u_r . In the electrical circuit, this is equivalent **to removing a charge $q_r = C(\theta - u_r)$ from the capacitor**, or equivalently **adding a negative charge $-q_r$ to it**
- Therefore, the reset is equivalent **to adding a short current pulse**

$$I_r = -q_r \delta(t - t_i^{(f)})$$

at the firing time $t_i^{(f)}$

- Hence, using Eq. (6), **the reset current** at time t is

$$I_r(t) = -q_r \sum_f \delta(t - t_i^{(f)}) = -C(\theta - u_r) S_i(t) \quad (7)$$

Arbitrary input current $I(t)$

- The **LIF model** is governed by Eq. (1), i.e.,

$$\tau_m \frac{du(t)}{dt} = -[u(t) - u_{\text{rest}}] + RI(t) \quad (1)$$

- In the presence of an **arbitrary time-dependent input current $I(t)$** , and without accounting for the firing model (threshold θ), it holds

$$u(t) = u_{\text{rest}} + \frac{R}{\tau_m} \int_0^{+\infty} \exp\left(-\frac{\xi}{\tau_m}\right) I(t - \xi) d\xi \quad (8)$$

- What happens if we add the firing behavior (threshold θ)?

LIF model revisited

arbitrary
input
current reset
current

- The input current $I(t)$ is updated as $I_{\text{tot}}(t) = I(t) + I_r(t)$
- And the differential equation governing the LIF becomes

$$\tau_m \frac{du(t)}{dt} = -[u(t) - u_{\text{rest}}] + R[I(t) + I_r(t)] \quad (1c)$$

$$\tau_m \frac{du(t)}{dt} = -[u(t) - u_{\text{rest}}] + RI(t) + RC(u_r - \theta) \sum_f \delta(t - t_i^{(f)})$$

$$\tau_m \frac{du(t)}{dt} = -[u(t) - u_{\text{rest}}] + \boxed{RI(t)} + \boxed{\tau_m(u_r - \theta) \sum_f \delta(t - t_i^{(f)})}$$

arbitrary input
current

$I_r(t)$ reset current

LIF differential eq with reset mechanism

- The final LIF model for the **neuron membrane evolution** is

$$\frac{du(t)}{dt} = -\frac{1}{\tau_m} [u(t) - u_{\text{rest}}] + \frac{RI(t)}{\tau_m} + (u_r - \theta) \sum_f \delta(t - t_i^{(f)}) \quad (9)$$

$$\frac{du(t)}{dt} = -\frac{1}{\tau_m} [u(t) - u_{\text{rest}}] + \frac{RI(t)}{\tau_m} + \underbrace{(u_r - \theta) S_i(t)}_{I_r(t)/\tau_m} \quad (10)$$

- This is **Eq. (7)** in reference paper [\[SPM-2019\]](#)
- [\[SPM-2019\]](#) Emre O. Neftci, Hesham Mostafa, and Friedemann Zenke, Surrogate Gradient Learning in Spiking Neural Networks: bringing the power of gradient-based optimization to spiking neural networks, *IEEE Signal Processing Magazine*, 2019.

Continuous time solution to the LIF model

- The solution to Eq. (9) is readily found by plugging $I_r(t)$ (Eq. (7)) into Eq. (8) and exploiting the linearity of the integral and the properties of the Dirac delta (when integrated)

$$u(t) = u_{\text{rest}} + (u_r - \theta) \sum_f \exp \left(-\frac{t - t_i^{(f)}}{\tau_m} \right) + \frac{R}{\tau_m} \int_0^{+\infty} \exp \left(-\frac{\xi}{\tau_m} \right) I(t - \xi) d\xi \quad (11)$$

- Where the firing times are defined by the threshold condition

$$t^{(f)} = \{t \mid u(t) = \theta\}$$

SPIKING NEURAL NETWORK TRAINING

Moving from Continuous to Discrete Time

Forward Euler method [ProcIEEE-2023]

- To make the previous equations useful for online learning we need to move into their **discrete time representation**
- The continuous time model of a LIF neuron is

$$\tau_m \frac{du(t)}{dt} = -[u(t) - u_{\text{rest}}] + RI_{\text{tot}}(t)$$

- Without loss of generality (R can be included in the SNN weights, *to be learned* and u_{rest} is a rescaling), we set:
 - $R=1$, $u_{\text{rest}}=0$
- **This allows to approximate (1) as (without taking the limit for $\Delta t \rightarrow 0$)**

$$\tau_m \frac{u(t + \Delta t) - u(t)}{\Delta t} = -u(t) + I_{\text{tot}}(t) \quad (12)$$

Forward Euler method [ProcIEEE-2023]

- Rewriting the terms of the last Eq. (12), we get

$$u(t + \Delta t) = \left(1 - \frac{\Delta t}{\tau_m}\right) u(t) + \frac{\Delta t}{\tau_m} I_{\text{tot}}(t)$$

- By defining the **voltage decaying factor** as

$$\beta := 1 - \frac{\Delta t}{\tau_m}$$

- We obtain (still continuous time)

$$u(t + \Delta t) = \beta u(t) + (1 - \beta) I_{\text{tot}}(t)$$

Characterizing β

- Assuming that $I_{\text{tot}}(t)=0$ (no input current), we get

$$u(t + \Delta t) = \beta u(t) + 0 \Rightarrow \beta = \frac{u(t + \Delta t)}{u(t)}$$

- This is the **decay coefficient for the membrane potential**
- In the absence of the input current, we have (u_0 is the membrane potential at time $t_0=0$)

$$u(t) = u_0 \exp \left(-\frac{t}{\tau_m} \right)$$

- And we obtain:

$$\beta = \frac{u(t + \Delta t)}{u(t)} = \exp \left(-\frac{\Delta t}{\tau_m} \right) \quad (13)$$

A note on $I_{\text{tot}}(t)$

- For the total input current, it holds

$$\begin{aligned} I_{\text{tot}}(t) &= I(t) + I_r(t) = I(t) + \tau_m(u_r - \theta)S_i(t) \\ &= I(t) - \tau_m\theta S_i(t) \end{aligned}$$

additionally assuming $u_r=0$

- It is customary to express the output spike train in discrete time n , $S_i[n]$ as

$$S_i[n] := \Theta(u_i[n] - \theta) \quad (14)$$

Heaviside step function

- That is, $S_i[n]$ is 1 when the potential reaches the **threshold θ**

Membrane potential: discrete model

- Now, assuming that time t is discretized into subsequential time steps $\Delta t = 1$, and referring to the discrete time as n (to emphasize discrete dynamics), we get

$$u[n + 1] = \beta u[n] + (1 - \beta) I_{\text{tot}}[n]$$

- Hence, for neuron i , we get

$$\begin{aligned} u_i[n + 1] &= \beta u_i[n] + (1 - \beta) (I_i[n] - \tau_m \theta S_i[n]) = \\ &= \beta u_i[n] + \frac{\Delta t}{\tau_m} (I_i[n] - \tau_m \theta S_i[n]) \stackrel{(\Delta t = 1)}{=} \\ &= \beta u_i[n] + (1 - \beta) I_i[n] - \theta S_i[n] \end{aligned}$$

- $(1 - \beta)$ is absorbed into the network weights W_{ij} (see next)

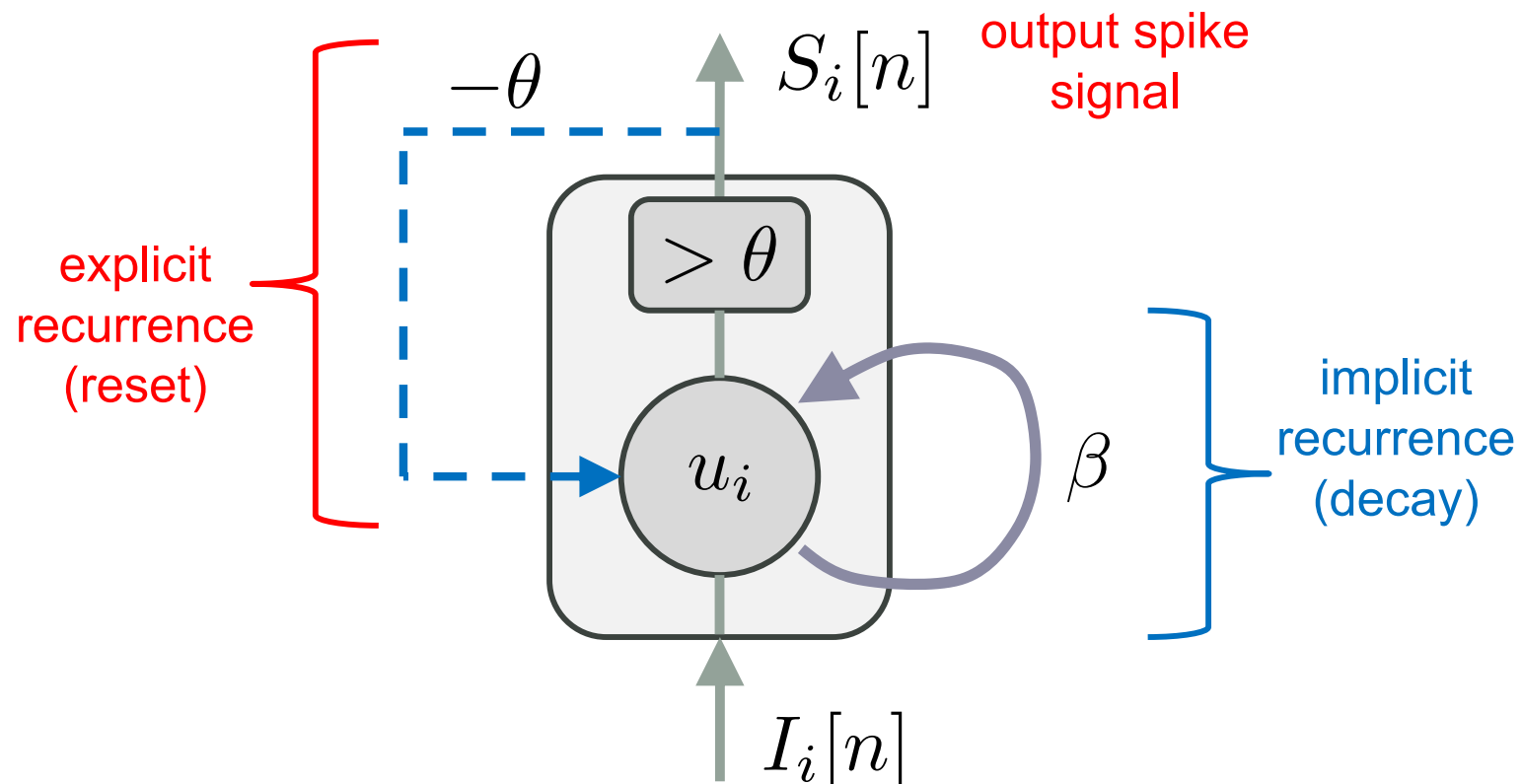
SNNs are inherently recurrent

- SNNs are **stateful**
 - Even if they do not have explicit recurrent connections
 - This is because neurons have an **internal state**

$$u_i[n + 1] = \underbrace{\beta u_i[n]}_{\text{decay}} + \underbrace{I_i[n]}_{\text{input}} - \underbrace{\theta S_i[n]}_{\text{reset}} \quad (15)$$

Recurrent nature of neuron dynamics

$$u_i[n + 1] = \underbrace{\beta u_i[n]}_{\text{decay}} + \underbrace{I_i[n]}_{\text{input}} - \underbrace{\theta S_i[n]}_{\text{reset}}$$



Discrete spiking neuron model

- Hence for $\begin{cases} \text{node } i \\ \text{layer } \ell \end{cases}$, we track the **state pair** $\begin{cases} u_i^{(\ell)}[n] \\ I_i^{(\ell)}[n] \end{cases}$

$$\begin{cases} u_i^{(\ell)}[n+1] = \beta u_i^{(\ell)}[n] + I_i^{(\ell)}[n] - \theta S_i^{(\ell)}[n] \\ I_i^{(\ell)}[n] = \sum_j W_{ij}^{(\ell)} S_j^{(\ell-1)}[n] \end{cases} \quad (16)$$

j
 spike signal from neurons in
 previous layer
 (feedforward part)

For a feed-forward SNN

- Equations for neuron i and layer ℓ are:

$$I_i^{(\ell)}[n] = \sum_j W_{ij}^{(\ell)} S_j^{(\ell-1)}[n]$$

$$u_i^{(\ell)}[n+1] = \beta u_i^{(\ell)}[n] + I_i^{(\ell)}[n] - \theta S_i^{(\ell)}[n]$$

$$S_i^\ell[n] := \Theta(u_i^\ell[n] - \theta)$$

For a feed-forward SNN

- Equations for neuron i and layer ℓ are:

$$I_i^{(\ell)}[n] = \sum_j W_{ij}^{(\ell)} S_j^{(\ell-1)}[n]$$

$$u_i^{(\ell)}[n] = \beta u_i^{(\ell)}[n-1] + I_i^{(\ell)}[n-1] - \theta S_i^{(\ell)}[n-1]$$

$$S_i^\ell[n] := \Theta(u_i^\ell[n] - \theta)$$

- Setting $n+1 \rightarrow n$ for the membrane equation

For a feed-forward SNN

- Equations for neuron i and layer ℓ are:

$$I_i^{(\ell)}[n] = \sum_j W_{ij}^{(\ell)} S_j^{(\ell-1)}[n]$$

$$u_i^{(\ell)}[n] = \beta u_i^{(\ell)}[n-1] + I_i^{(\ell)}[n] - \theta S_i^{(\ell)}[n-1]$$

$$S_i^\ell[n] := \Theta(u_i^\ell[n] - \theta)$$

- Typically:** the time index of the input current **is time-shifted by one unit** so that it can **instantaneously contribute to the membrane potential**. **This is not physiologically precise**, but it is preferred to cast the SNN model into a standard RNN model

First standard LIF model & notation

- Often, $I \rightarrow X, u \rightarrow H$

Input current

$$X_i^{(\ell)}[n] = \sum_j W_{ij}^{(\ell)} S_j^{(\ell-1)}[n]$$

Membrane potential update

$$H_i^{(\ell)}[n] = \beta H_i^{(\ell)}[n-1] + X_i^{(\ell)}[n] - \theta S_i^{(\ell)}[n-1]$$

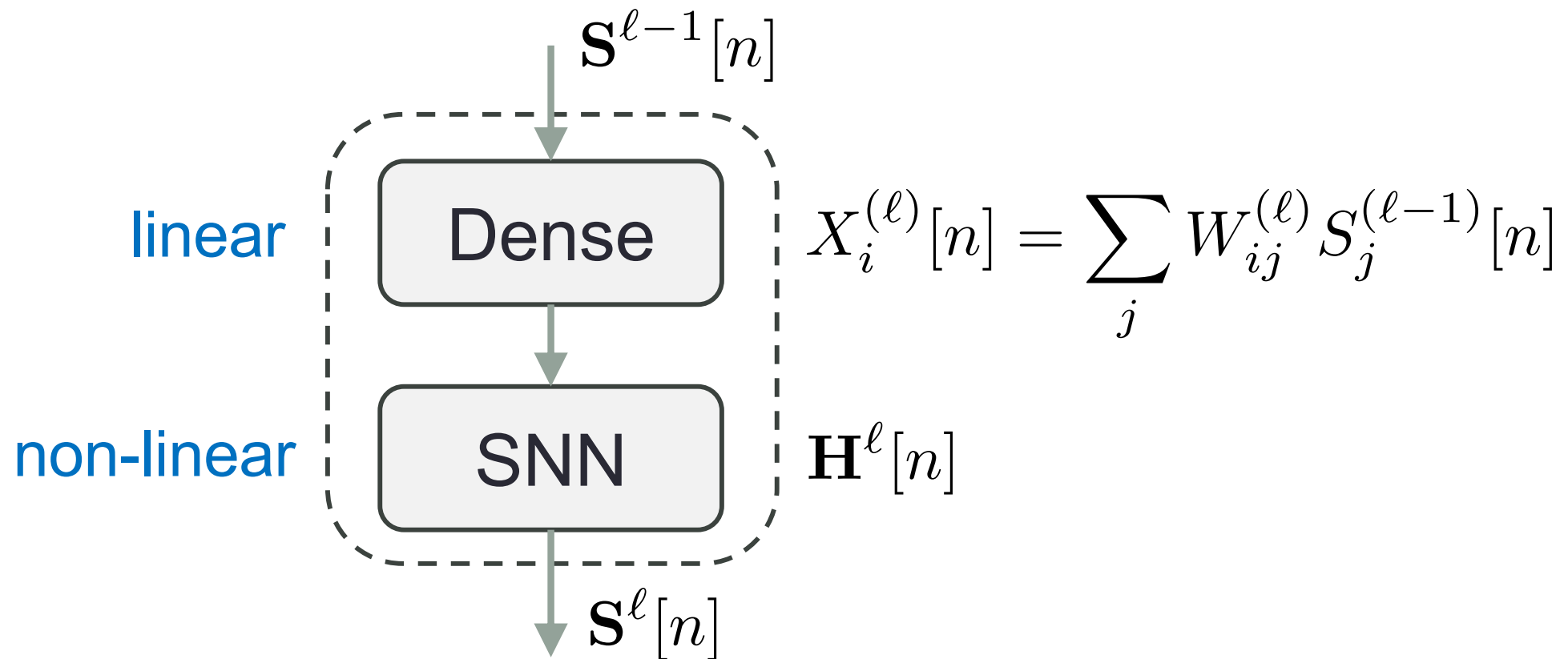
Neuron output

$$S_i^\ell[n] := \Theta(H_i^\ell[n] - \theta)$$

- Here, the reset mechanism acts by subtracting a fixed quantity to the membrane potential

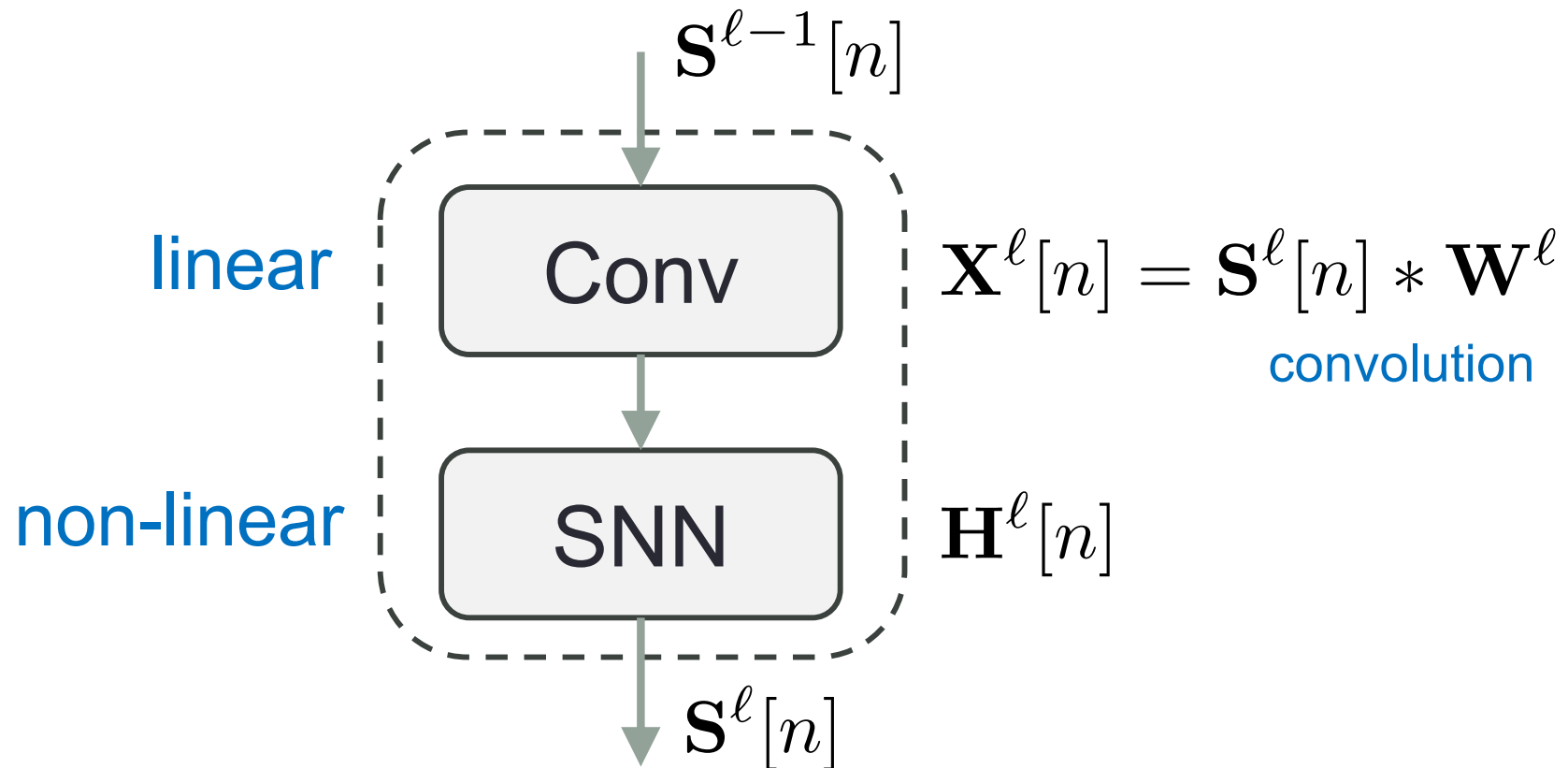
First standard LIF model

- Often, with existing frameworks (e.g., SNNtorch)



First standard LIF model

- The dense layer can be replaced with a convolutional layer



Second standard LIF model

We may differentiate the membrane potential after the neuronal dynamics $H[n]$ and after the triggering of a spike $V[n]$

$$X_i^{(\ell)}[n] = \sum_j W_{ij}^{(\ell)} S_j^{(\ell-1)}[n] \quad \text{input current}$$

$$H_i^{(\ell)}[n] = \beta V_i^{(\ell)}[n-1] + X_i^{(\ell)}[n] \quad \text{neuronal dynamics}$$

$$S_i^\ell[n] := \Theta(H_i^\ell[n] - \theta) \quad \text{spiking behavior}$$

$$V_i^{(\ell)}[n] = H_i^{(\ell)}[n](1 - S_i^\ell[n]) + V_{\text{reset}} S_i^\ell[n]$$

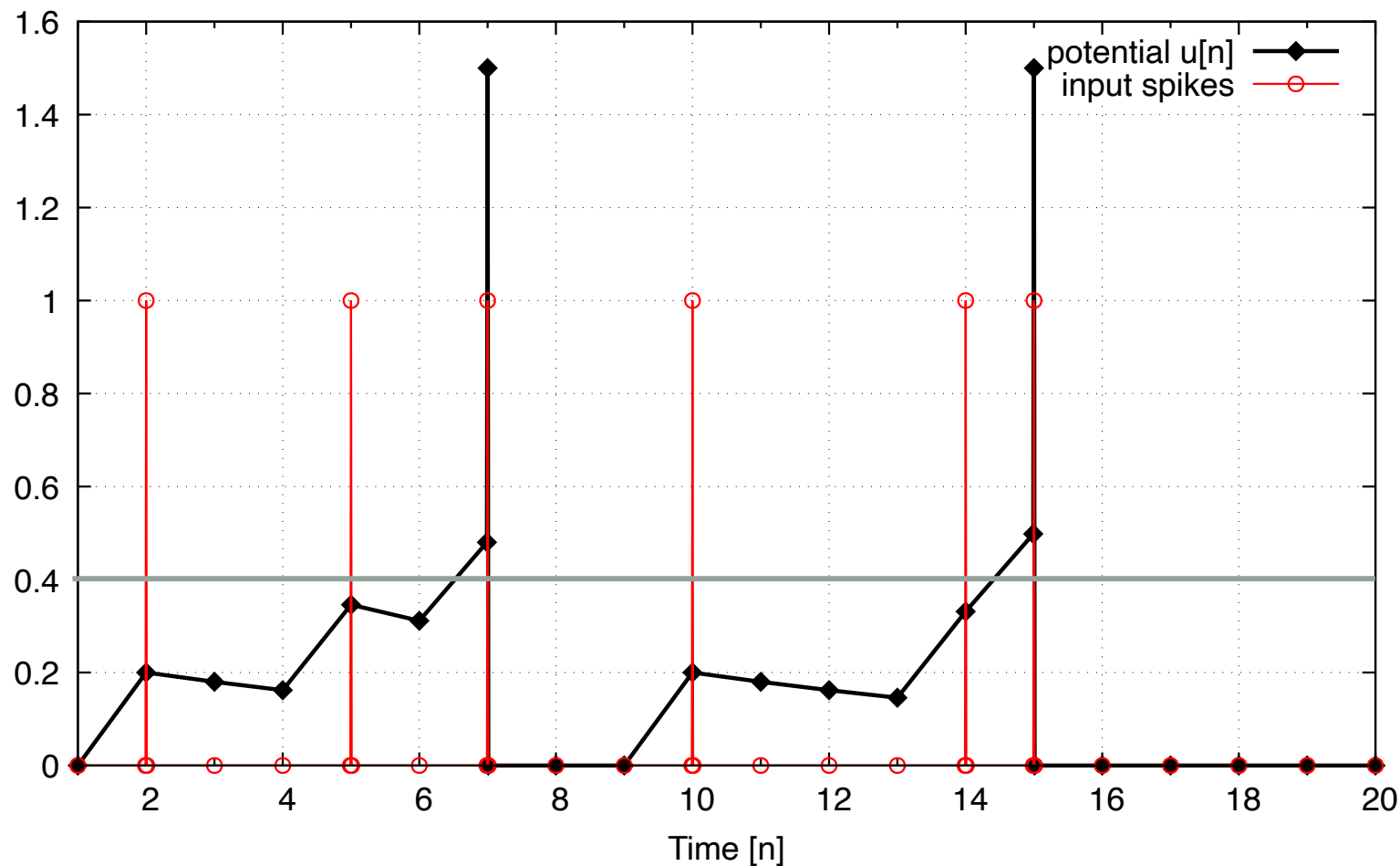
explicit reset mechanism (with **tunable reset potential**)

Spiking neuron – example 1

$$\beta = 0.9$$

$$\theta = 0.4$$

$$W = 0.2$$



$$\theta = 0.4$$

$$u_{\text{ref}} = 0$$

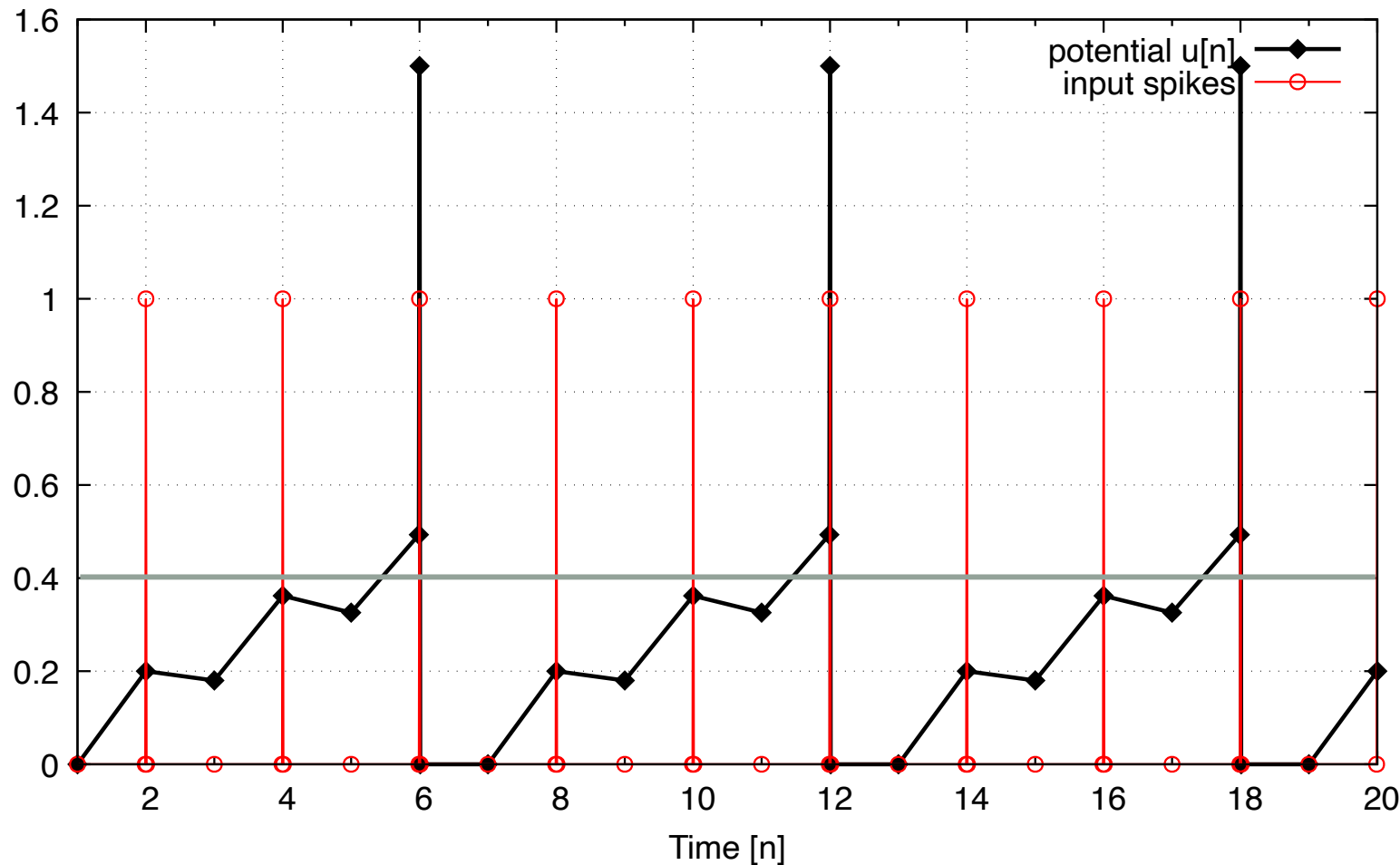
$$u_r = 0$$

Spiking neuron – example 2

$$\beta = 0.9$$

$$\theta = 0.4$$

$$W = 0.2$$



$$\theta = 0.4$$

$$u_{\text{ref}} = 0$$

$$u_r = 0$$

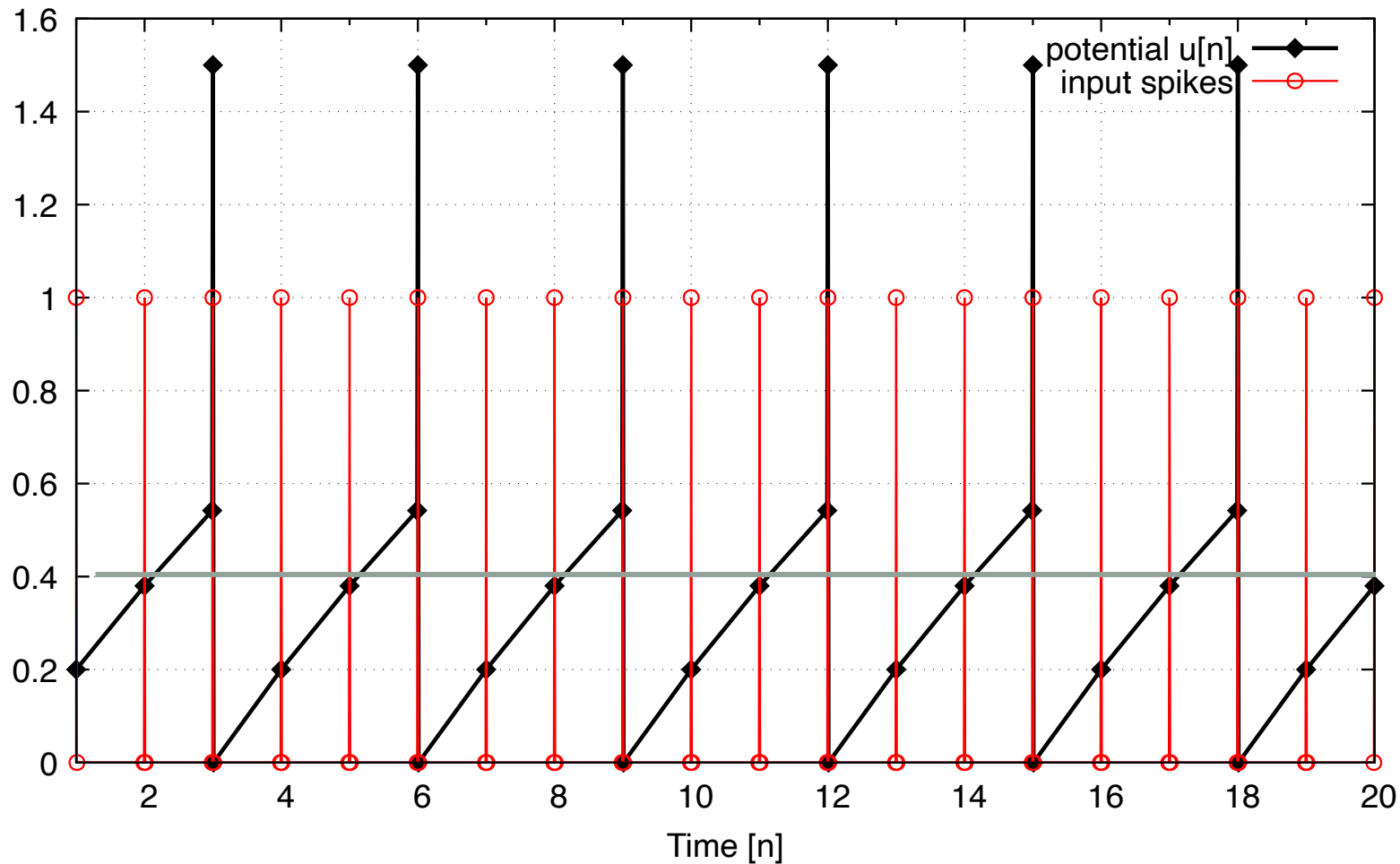
Spiking neuron – example 3

$$\beta = 0.9$$

$$\theta = 0.4$$

$$W = 0.2$$

constant input current



$$\theta = 0.4$$

$$u_{\text{ref}} = 0$$

$$u_r = 0$$

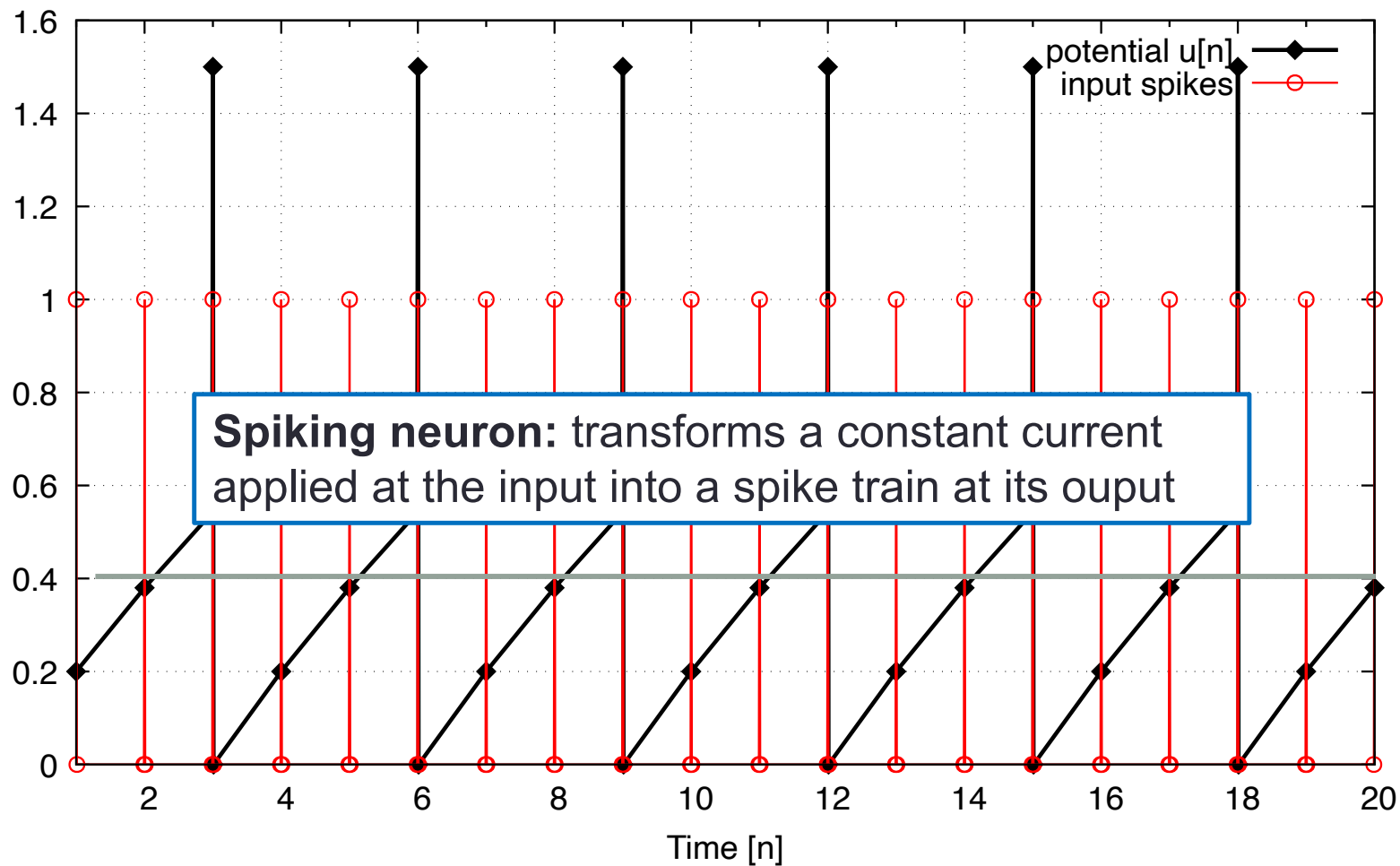
Spiking neuron – example 3

$$\beta = 0.9$$

$$\theta = 0.4$$

$$W = 0.2$$

constant input current



$$\theta = 0.4$$

$$u_{\text{ref}} = 0$$

$$u_r = 0$$

SPIKING NEURAL NETWORK TRAINING

Backpropagation – Surrogate Gradient Descent

Spiking SNN: training problem

Input current

$$X_i^{(\ell)}[n] = \sum_j W_{ij}^{(\ell)} S_j^{(\ell-1)}[n]$$

Membrane potential update

$$H_i^{(\ell)}[n] = \beta H_i^{(\ell)}[n-1] + X_i^{(\ell)}[n] - \theta S_i^{(\ell)}[n-1]$$

Neuron output

$$S_i^\ell[n] := \Theta(H_i^\ell[n] - \theta)$$

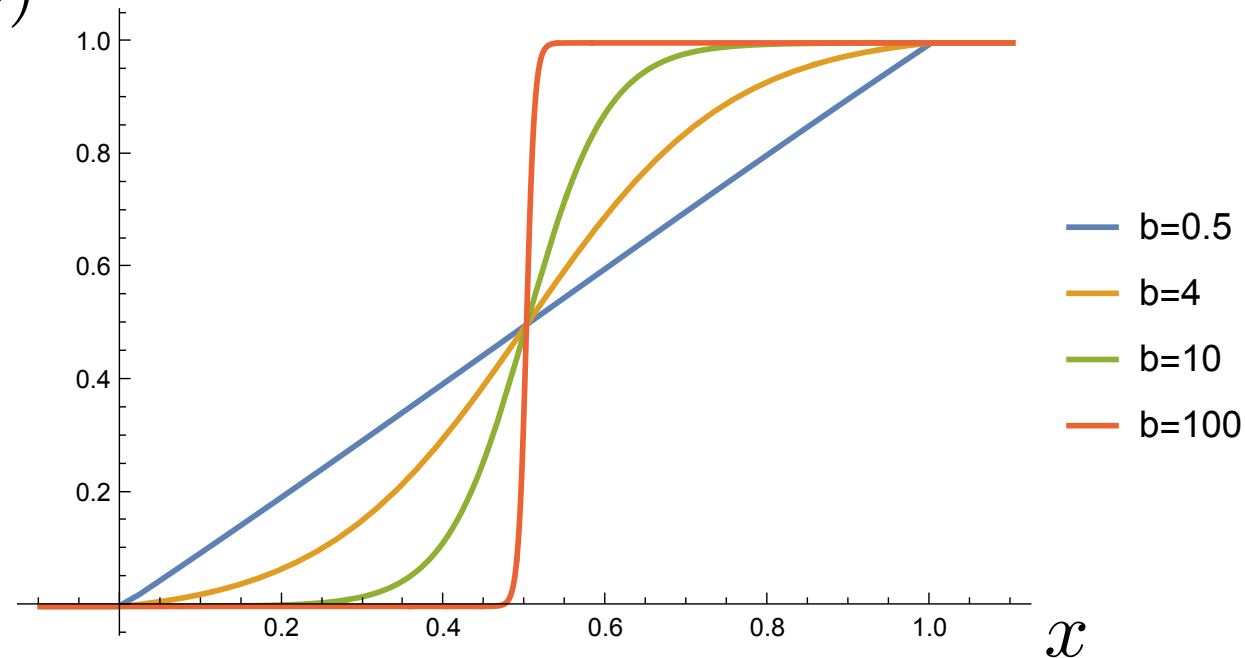
- **Problem:** Heaviside step function is **non-differentiable**
 - The gradient is zero everywhere except when $H_i^\ell[n]$ equals the threshold θ , for which it is a Dirac delta. Gradient backpropagation cannot be used directly

Surrogate gradient for SNN training

- 1) Use the **Heaviside function** for the forward pass
 - Propagate DATA from input → output
- 2) **Unfold the SNN** through time – obtaining a DAG
 - As you would do for a standard Recurrent Neural Network (RNN)
- 3) **Approximate the Heaviside** function during the gradient backpropagation using a **surrogate function**
 - The surrogate function has a proper derivative everywhere
- 4) Several surrogate functions can be used
 - Learning is reasonably robust to such approximations
 - In this way, **standard backpropagation** can be used (usual chain rule of derivatives for functions defined on DAGs)

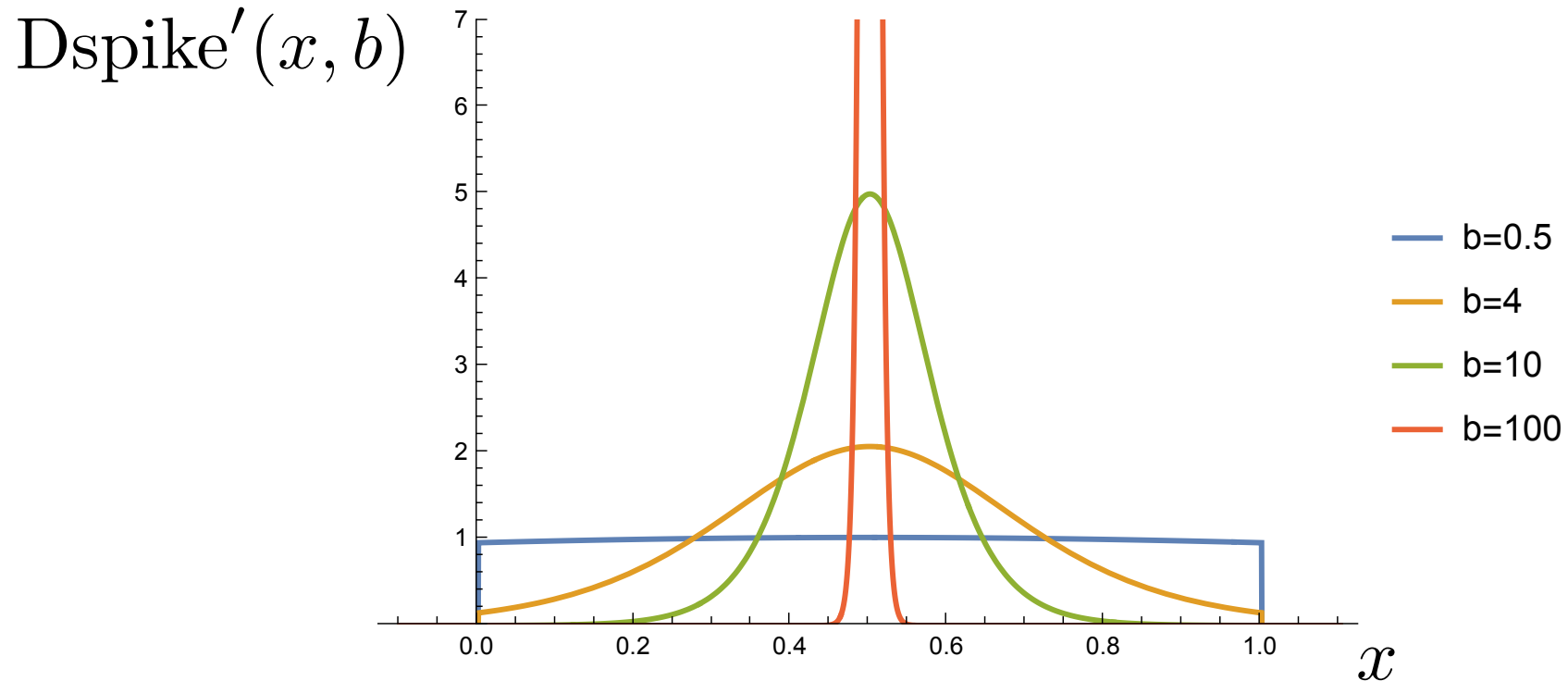
Parameterized smoothing function

$\text{Dspike}(x, b)$



$$\text{Dspike}(x, b) = \begin{cases} 0 & x < 0 \\ \frac{\tanh(b(x-0.5)) + \tanh(b/2)}{2 \tanh(b/2)} & 0 \leq x \leq 1 \\ 1 & x > 1 \end{cases}$$

Smoothing function derivative

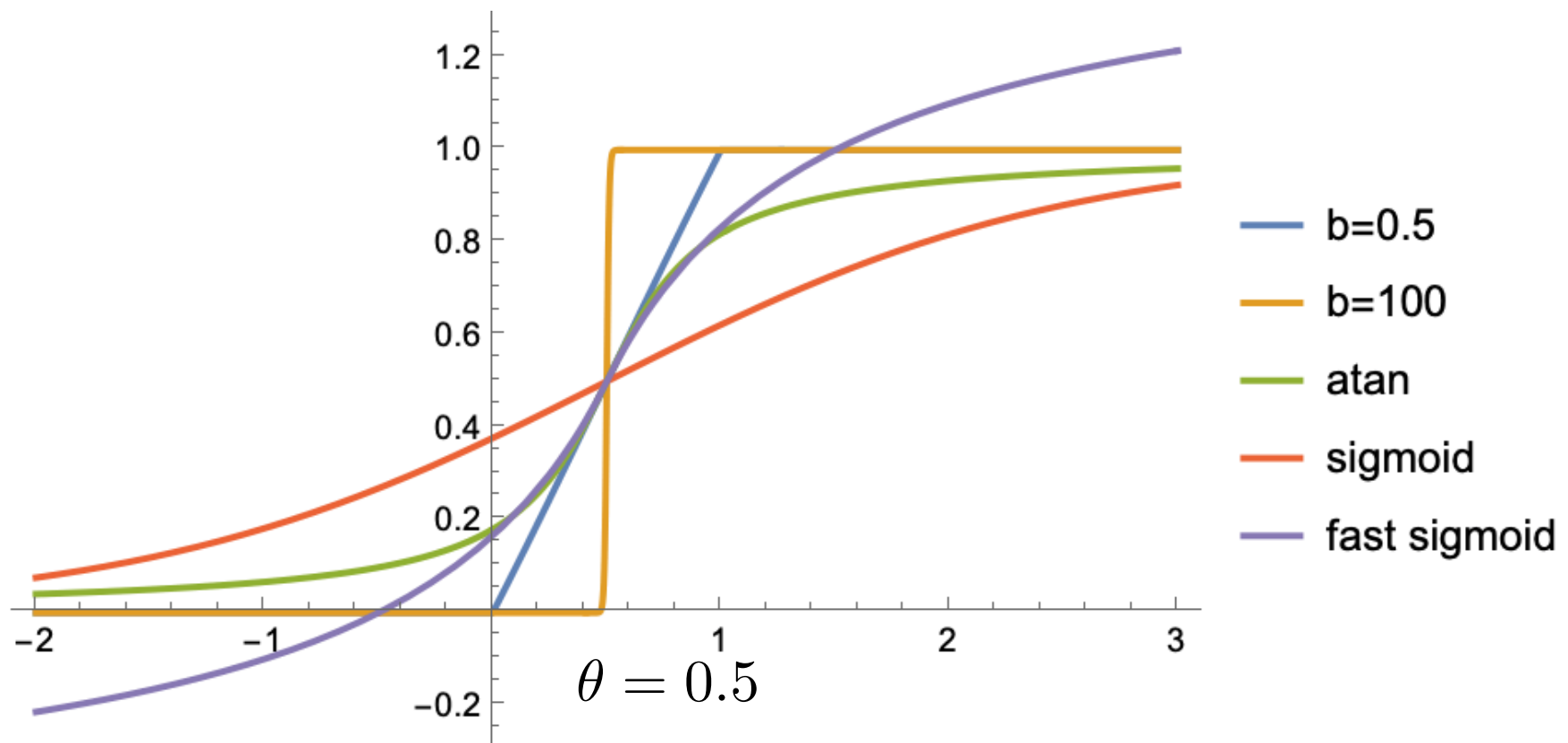


$$D\text{spike}'(x, b) = \frac{a}{2} \coth\left(\frac{a}{2}\right) \text{sech}\left[a(y - 0.5)\right]^2$$

Typical surrogate gradient functions

$\frac{1}{\pi} \arctan(\pi x)$	arctan
$\frac{1}{1 + \exp(-x)}$	sigmoid
$\frac{x}{1 + x }$	fast sigmoid
$\text{Dspike}(b, x)$	dspike

Typical surrogate gradient functions



Feed-forward SNN model

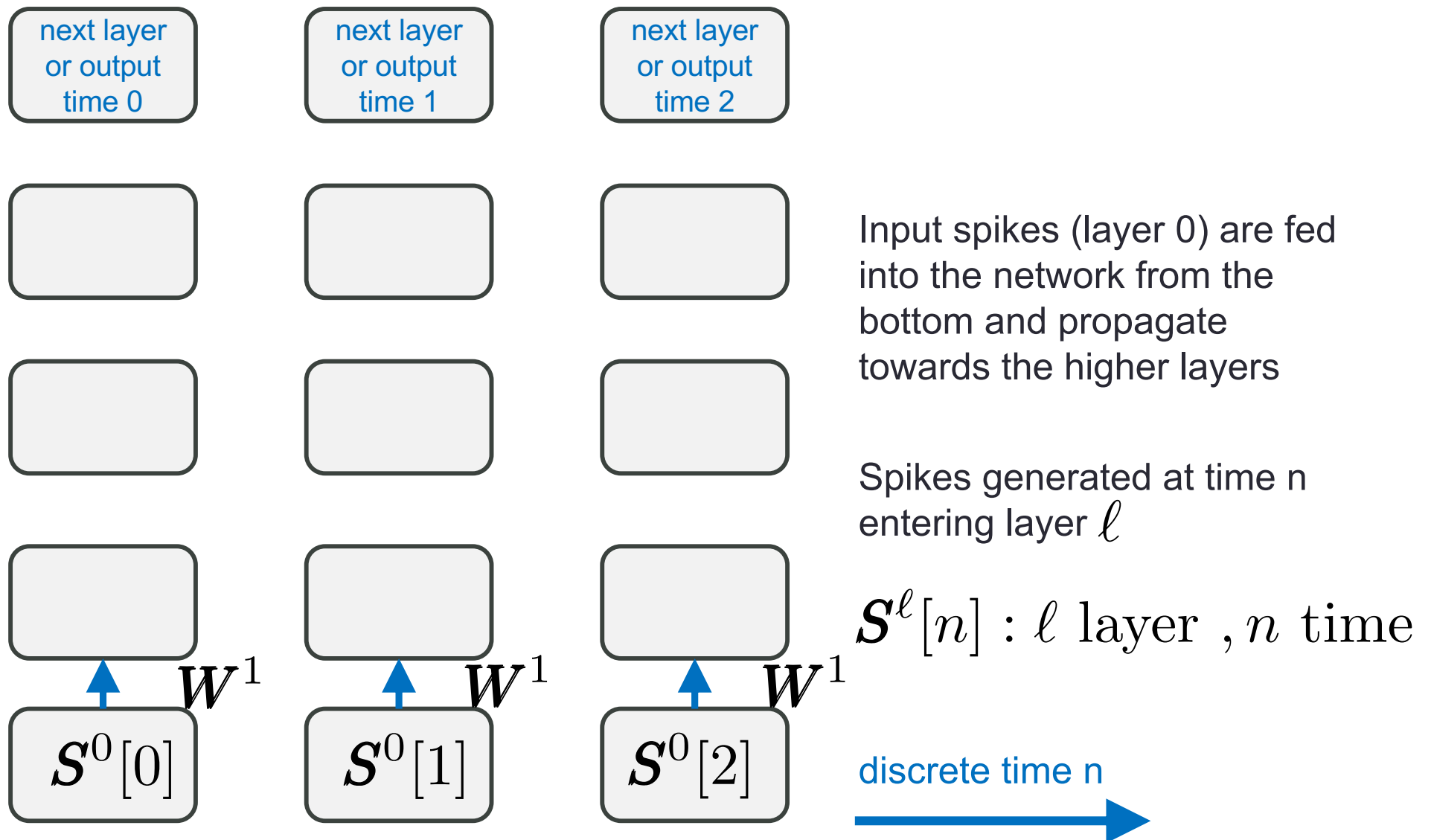
- Equations for neuron i and layer ℓ are:

$$I_i^{(\ell)}[n] = \sum_j W_{ij}^{(\ell)} S_j^{(\ell-1)}[n]$$

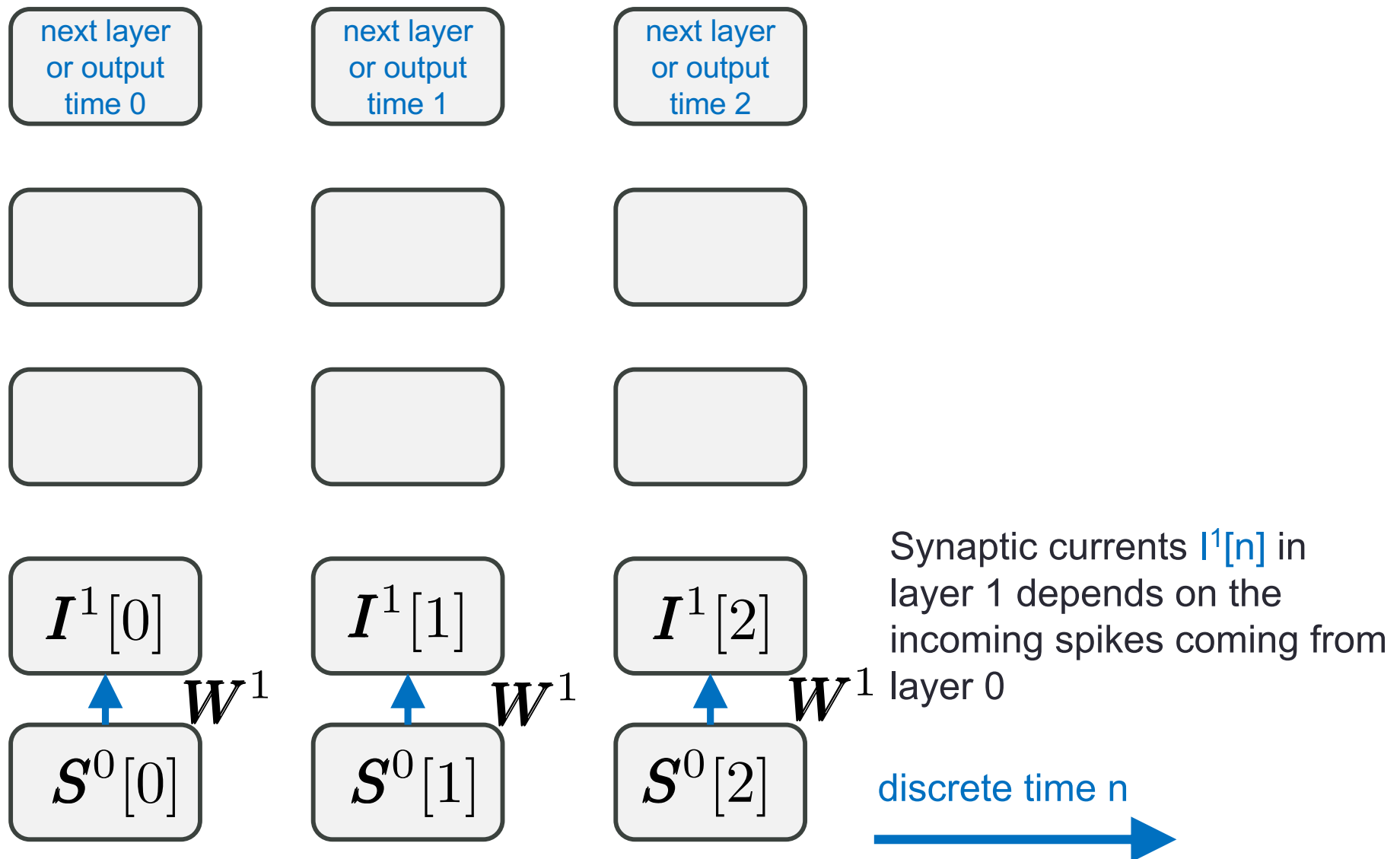
$$u_i^{(\ell)}[n] = \beta u_i^{(\ell)}[n-1] + I_i^{(\ell)}[n-1] - \theta S_i^{(\ell)}[n-1]$$

$$S_i^\ell[n] := \Theta(u_i^\ell[n] - \theta)$$

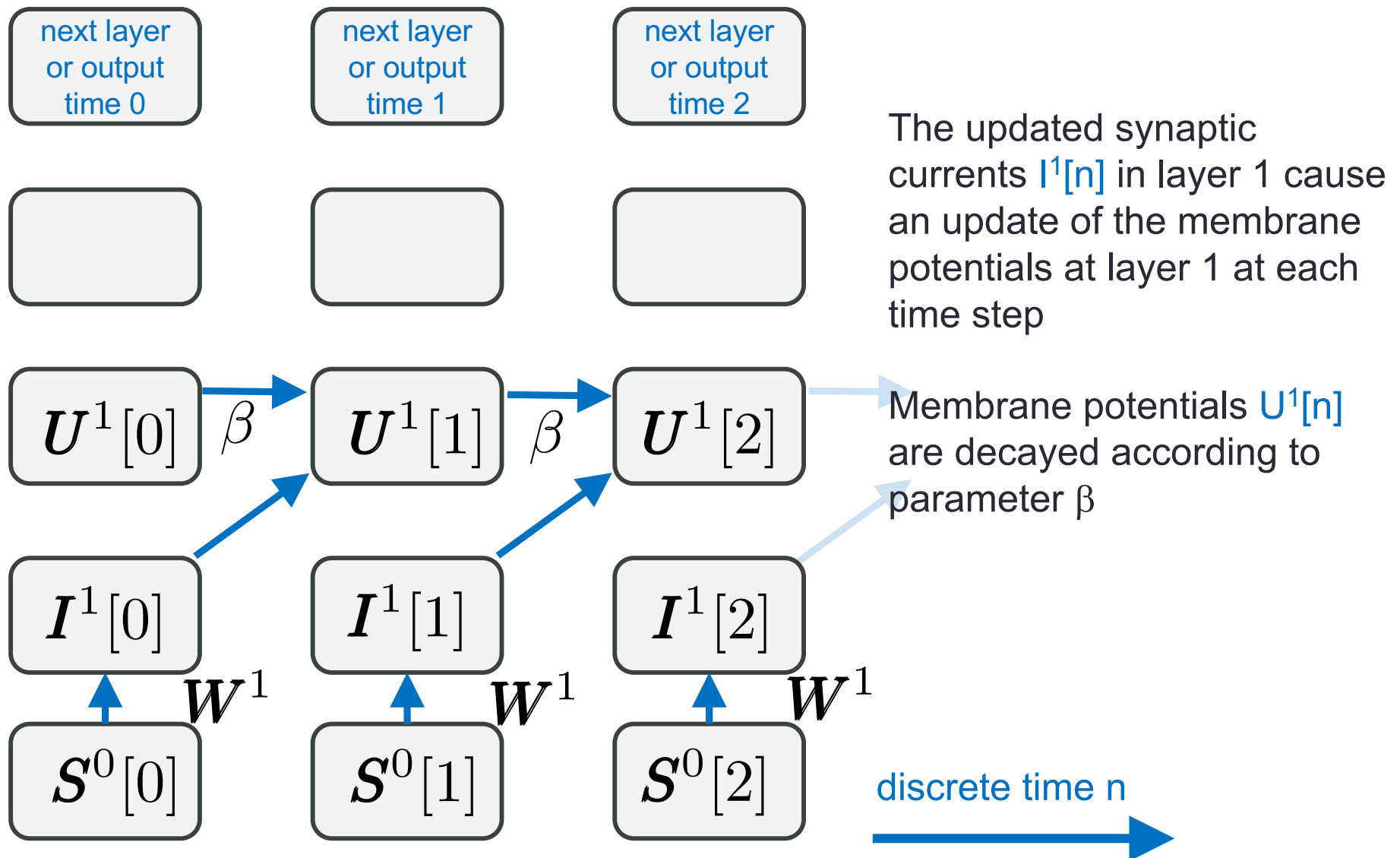
SNN computational graph (unrolled)



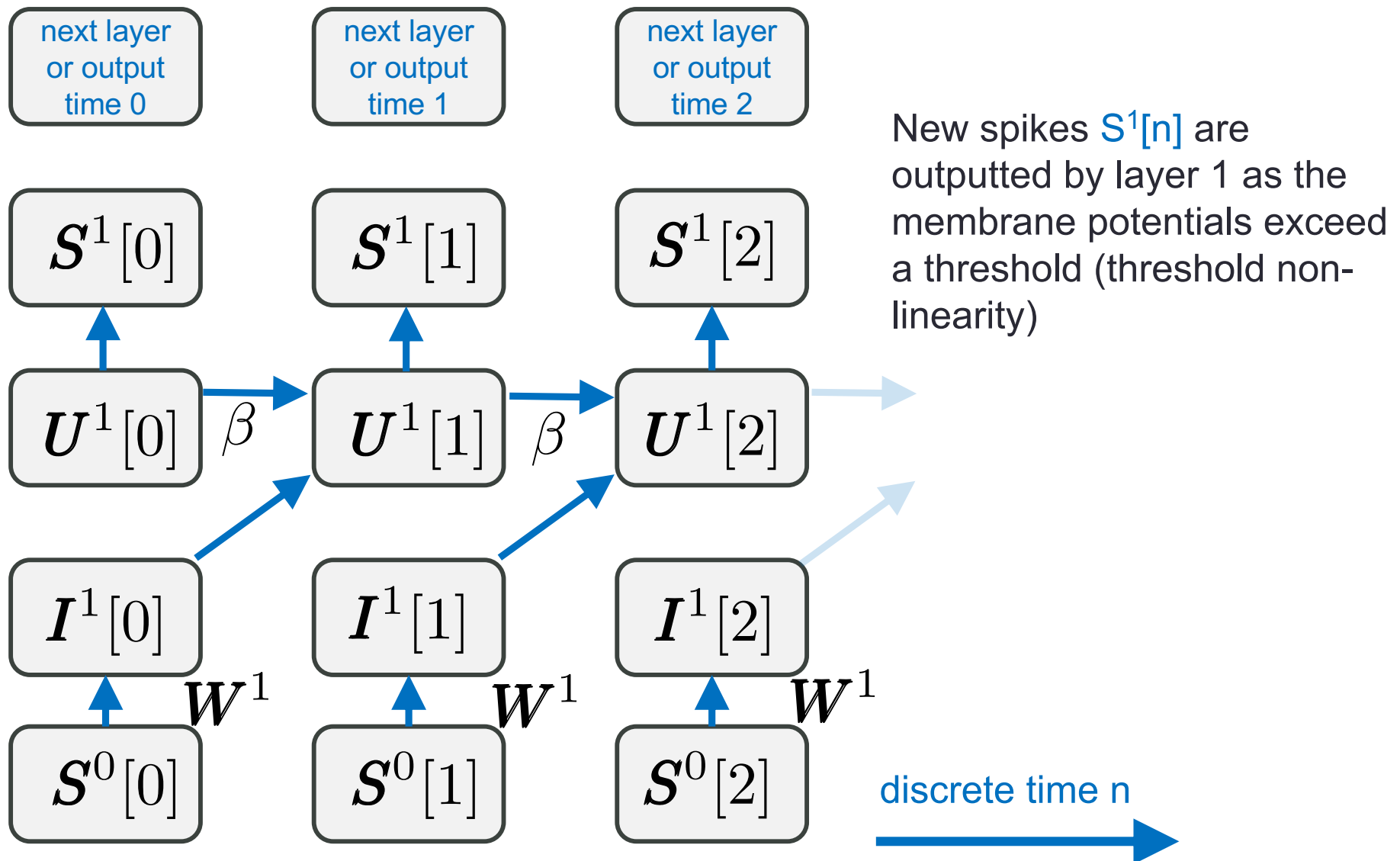
SNN computational graph (unrolled)



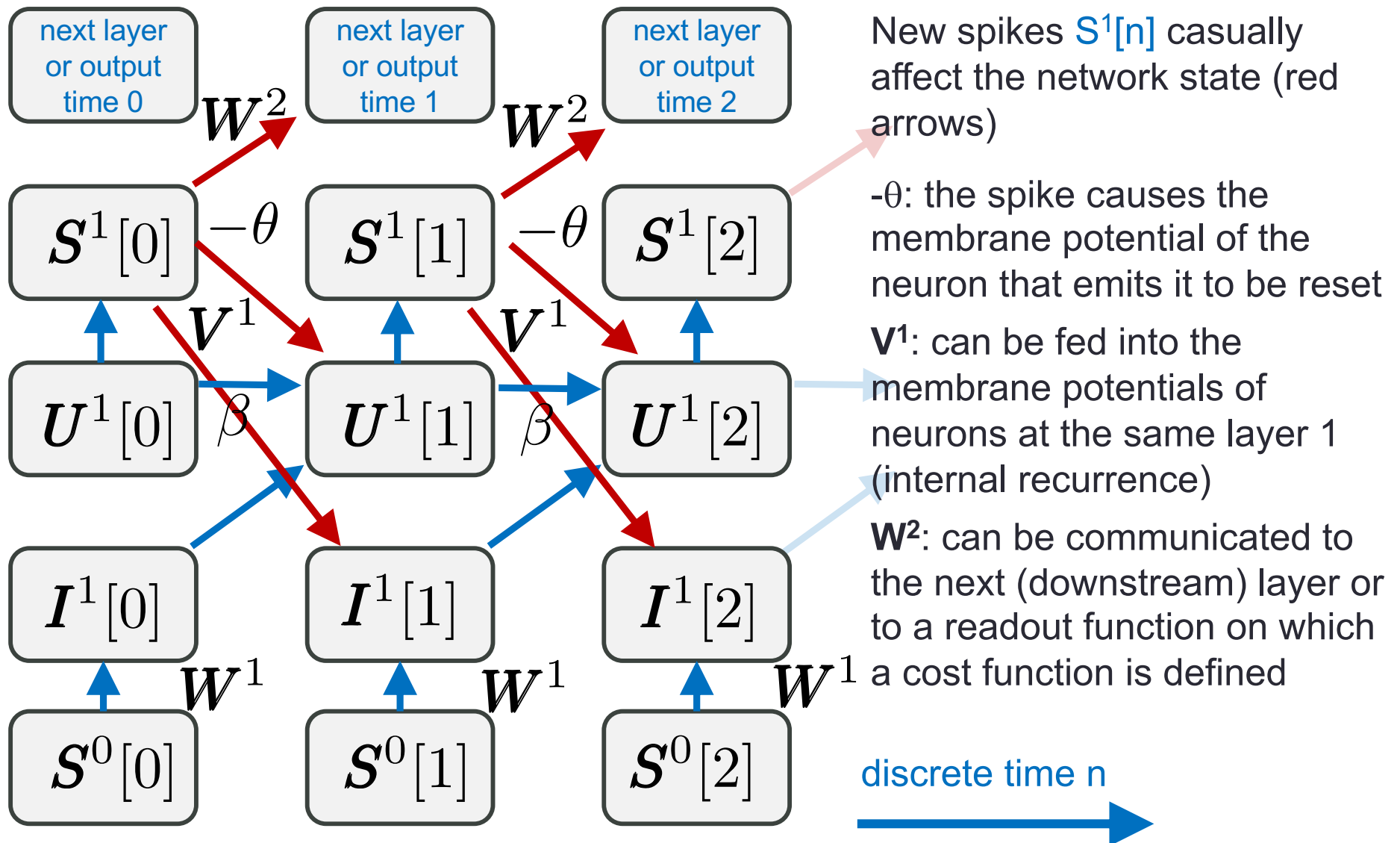
SNN computational graph (unrolled)



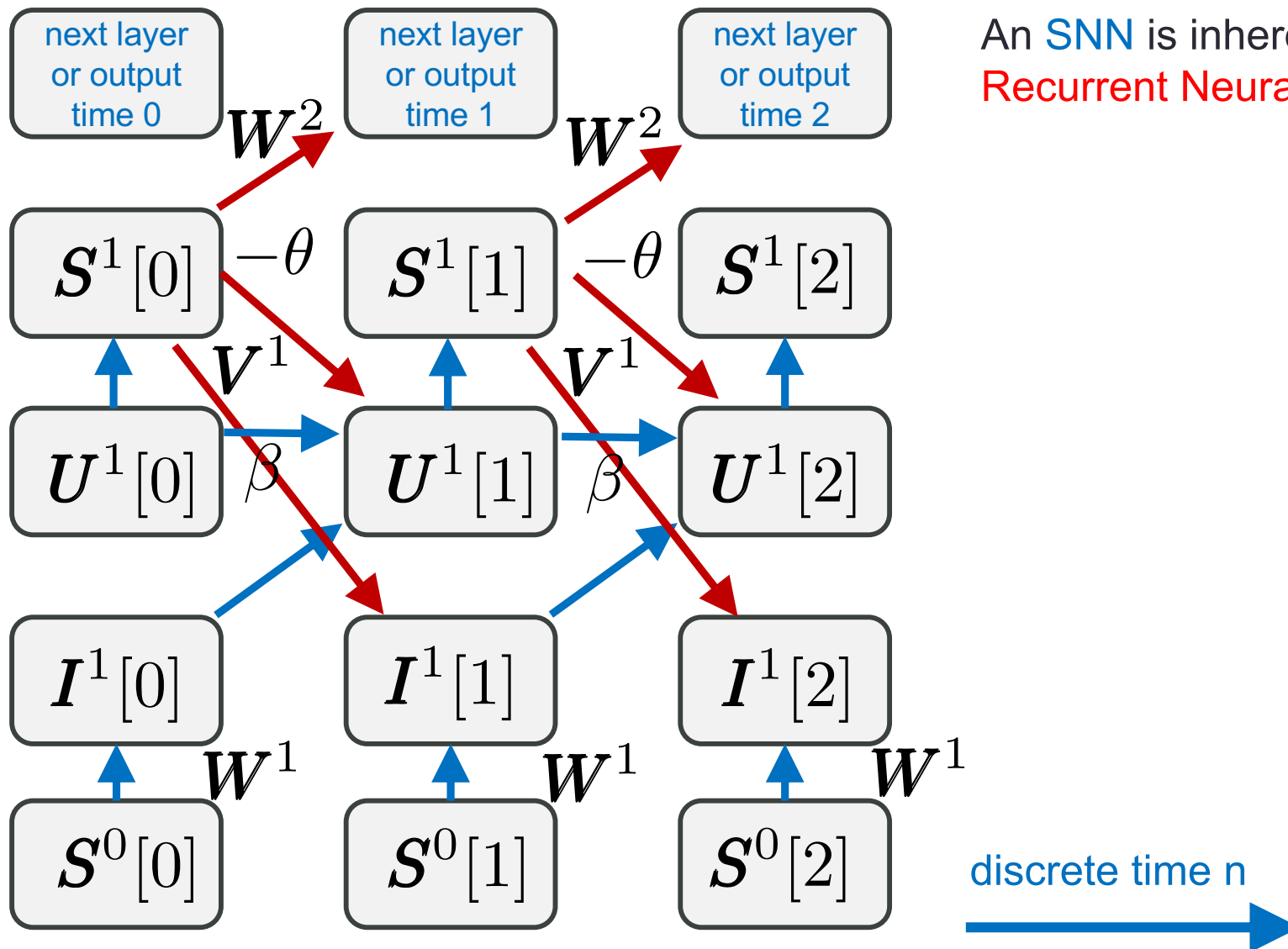
SNN computational graph (unrolled)



SNN computational graph (unrolled)

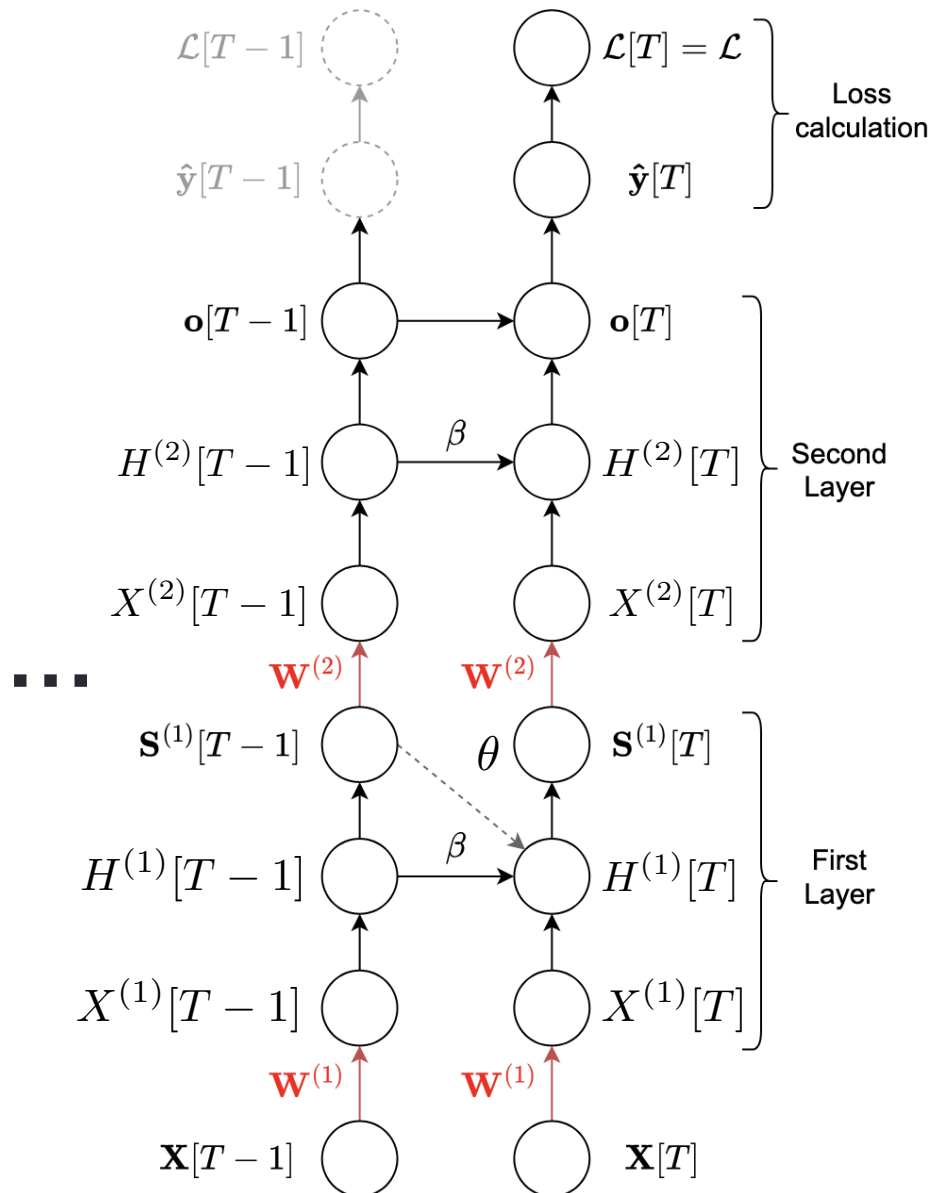


SNN computational graph (unrolled)



An SNN is inherently a
Recurrent Neural Network

Two-layer SNN



$$\mathbf{X}^{(1)}[t] = \mathbf{W}^{(1)} \mathbf{X}[t]$$

$$\mathbf{H}^{(1)}[t] = \beta \mathbf{H}^{(1)}[t-1] + \mathbf{X}^{(1)}[t] - \theta \mathbf{S}^{(1)}[t-1] \quad \text{reset}$$

$$\mathbf{S}^{(1)}[t] = \Theta(\mathbf{H}^{(1)}[t] - \theta)$$

$$\mathbf{X}^{(2)}[t] = \mathbf{W}^{(2)} \mathbf{S}^{(1)}[t]$$

$$\mathbf{H}^{(2)}[t] = \beta \mathbf{H}^{(2)}[t-1] + \mathbf{X}^{(2)}[t]$$

$$\mathbf{o}[t] = \sum_{j \leq t} \mathbf{H}^{(2)}[j]$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{o}[t])$$

$$\mathcal{L} = \mathcal{L}[T] = - \sum_{k=1}^K y_k \log(\hat{y}_k[T])$$

$$= - \langle \mathbf{y}, \log(\mathbf{y}[T]) \rangle \quad \text{categorical cross-entropy}$$

Loss is computed in the last time step T
All vars at time t=0 are 0

Gradient descent – layer 2

Output layer

- Using the chain rule of derivatives for the output Layer 2, we have:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(2)}} = \frac{\partial \mathcal{L}}{\partial o_i[T]} \frac{\partial o_i[T]}{\partial W_{ij}^{(2)}}$$

- Since

$$o_i[T] = \sum_{t=1}^T H_i^{(2)}[t] \quad \Rightarrow \quad \frac{\partial o_i[T]}{\partial W_{ij}^{(2)}} = \sum_{t=1}^T \frac{\partial H_i^{(2)}[t]}{\partial W_{ij}^{(2)}}$$

Gradient descent – layer 2

- Starting from

$$\mathbf{H}^{(2)}[t] = \beta \mathbf{H}^{(2)}[t-1] + \mathbf{X}^{(2)}[t]$$

- We can develop the recursion:

$$\begin{aligned} H_i^{(2)}[t] &= \beta H_i^{(2)}[t-1] + I_i^{(2)}[t] = \beta^2 H_i^{(2)}[t-2] + \beta H_i^{(2)}[t-1] + X_i^{(2)}[t] = \\ &= \beta^2 H_i^{(2)}[t-2] + \beta X_i^{(2)}[t-1] + X_i^{(2)}[t] = \\ &= \beta^3 H_i^{(2)}[t-3] + \beta^2 X_i^{(2)}[t-2] + \beta X_i^{(2)}[t-1] + X_i^{(2)}[t] = \dots \\ &= \beta^{t-1} H_i^{(2)}[1] + \beta^{t-2} X_i^{(2)}[2] + \beta^{t-3} X_i^{(2)}[3] + \dots + \beta X_i^{(2)}[t-1] + X_i^{(2)}[t] \end{aligned}$$

$$H_i^{(2)}[t] = \sum_{k=1}^t \beta^{t-k} X_i^{(2)}[k] \quad \Rightarrow \quad \frac{\partial H_i^{(2)}[t]}{\partial W_{ij}^{(2)}} = \sum_{k=1}^t \beta^{t-k} \frac{\partial X_i^{(2)}[k]}{\partial W_{ij}^{(2)}}$$

Gradient descent – layer 2

Layer 2

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(2)}} = \frac{\partial \mathcal{L}}{\partial o_i[T]} \frac{\partial o_i[T]}{\partial W_{ij}^{(2)}} = \frac{\partial \mathcal{L}}{\partial o_i[T]} \sum_{t=1}^T \frac{\partial H_i^{(2)}[t]}{\partial W_{ij}^{(2)}}$$

$$\frac{\partial \mathcal{L}}{\partial o_i[T]} = \hat{y}_i[T] - y_i$$

softmax

$$\frac{\partial H_i^{(2)}[t]}{\partial W_{ij}^{(2)}} = \sum_{k=1}^t \beta^{t-k} \frac{\partial X_i^{(2)}[k]}{\partial W_{ij}^{(2)}} = \sum_{k=1}^t \beta^{t-k} S_j^{(1)}[k]$$

Gradient descent – layer 2

- Hence, we get

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(2)}} = (\hat{y}_i[T] - y_i) \sum_{t=1}^T \sum_{k=1}^t \beta^{t-k} S_j^{(1)}[k]$$

Gradient descent – Layer 1

Layer 1

$$\begin{aligned}
 \frac{\partial \mathcal{L}}{\partial W_{ij}^{(1)}} &= \frac{\partial \mathcal{L}}{\partial o_i[T]} \frac{\partial o_i[T]}{\partial W_{ij}^{(1)}} = \\
 &= \frac{\partial \mathcal{L}}{\partial o_i[T]} \sum_{t=1}^T \frac{\partial H_i^{(2)}[t]}{\partial W_{ij}^{(1)}} = \text{similar recursion as for layer 2} \\
 &= \frac{\partial \mathcal{L}}{\partial o_i[T]} \sum_{t=1}^T \sum_{k=1}^t \beta^{t-k} \frac{\partial X_i^{(2)}[k]}{\partial W_{ij}^{(1)}}
 \end{aligned}$$

?

Gradient descent – layer 1

- For term $\frac{\partial X_i^{(2)}[k]}{\partial W_{ij}^{(1)}}$

- The following holds

$$\frac{\partial X_i^{(2)}[k]}{\partial W_{ij}^{(1)}} = W_{ii}^{(2)} \frac{\partial S_i^{(1)}[k]}{\partial W_{ij}^{(1)}}$$

$$\frac{d \arctan(x)}{dx} = \frac{1}{1+x^2}$$

↓
surrogate function (e.g., arctan)

$$\frac{\partial S_i^{(1)}[k]}{\partial W_{ij}^{(1)}} = \frac{\partial \tilde{S}_i^{(1)}[k]}{\partial H_i^{(1)}[k]} \frac{\partial H_i^{(1)}[k]}{\partial W_{ij}^{(1)}} = \frac{1}{1 + (\pi H_i^{(1)}[k])^2} \sum_{s=1}^k \beta^{k-s} \frac{\partial X_i^{(1)}[s]}{\partial W_{ij}^{(1)}}$$

Gradient descent – layer 1

- Putting it all together

$$\begin{aligned}
 \frac{\partial \mathcal{L}}{\partial W_{ij}^{(1)}} &= \frac{\partial \mathcal{L}}{\partial o_i[T]} \sum_{t=1}^T \sum_{k=1}^t \beta^{t-k} W_{ii}^{(2)} \frac{\partial \tilde{S}_i[k]}{\partial H_i^{(1)}[k]} \frac{\partial H_i^{(1)}[k]}{\partial W_{ij}^{(1)}} = \\
 &= (\hat{y}_i[t] - y_i) \sum_{t=1}^T \sum_{k=1}^t \beta^{t-k} \frac{W_{ii}^{(2)}}{1 + (\pi H_i^{(1)}[k])^2} \sum_{s=1}^k \beta^{k-s} X_j[s]
 \end{aligned}$$

vanishes for increasing H

Gradient descent – weight adaptation

Layer 2

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(2)}} = (\hat{y}_i[T] - y_i) \sum_{t=1}^T \sum_{k=1}^t \beta^{t-k} S_j^{(1)}[k]$$

Layer 1

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(1)}} = (\hat{y}_i[t] - y_i) \sum_{t=1}^T \sum_{k=1}^t \beta^{t-k} \frac{W_{ii}^{(2)}}{1 + (\pi H_i^{(1)}[k])^2} \sum_{s=1}^k \beta^{k-s} X_j[s]$$

derivative of arctan (surrogate function)
 vanishes for increasing H

Gradient descent

Layer 2

$$\frac{\partial \mathcal{L}}{\partial W^{(2)}} = (\hat{y}_i[T] - y_i) \sum_{t=1}^T \sum_{k=1}^t \beta^{t-k} S_j^{(1)}[k]$$

Surrogate gradients lead to vanishing gradients when multiple layers are stacked!

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(1)}} = (\hat{y}_i[t] - y_i) \sum_{t=1}^T \sum_{k=1}^t \beta^{t-k} \frac{W_{ii}^{(2)}}{1 + (\pi H_i^{(1)}[k])^2} \sum_{s=1}^k \beta^{k-s} X_j[s]$$

derivative of arctan (surrogate function)

May be smaller than 1 for increasing H

Training example: dataset

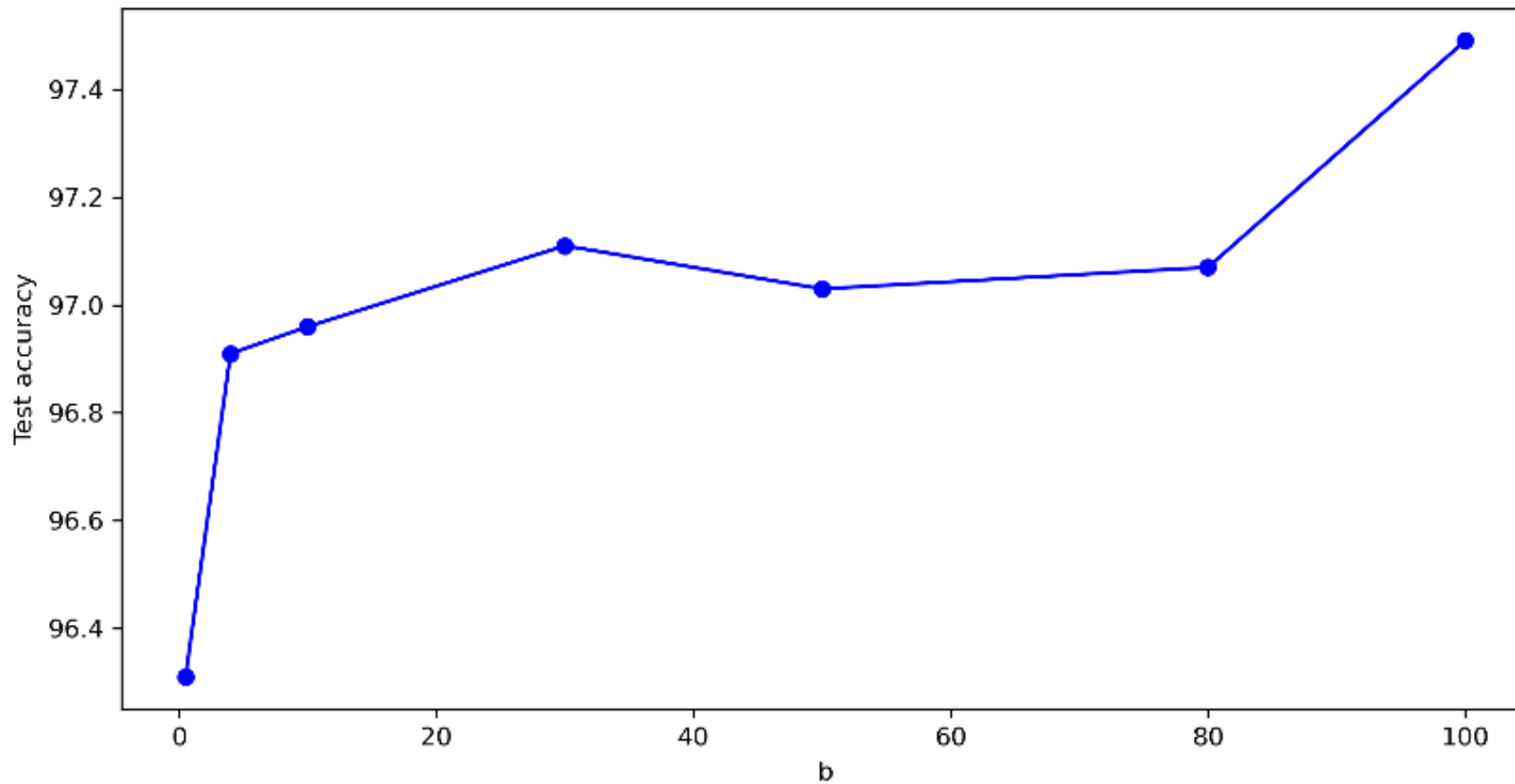
MNIST

The MNIST database of [handwritten digits](#) is a well-known dataset in the field of image recognition. It consists of 60,000 training samples and 10,000 test samples of 28x28 resolution.

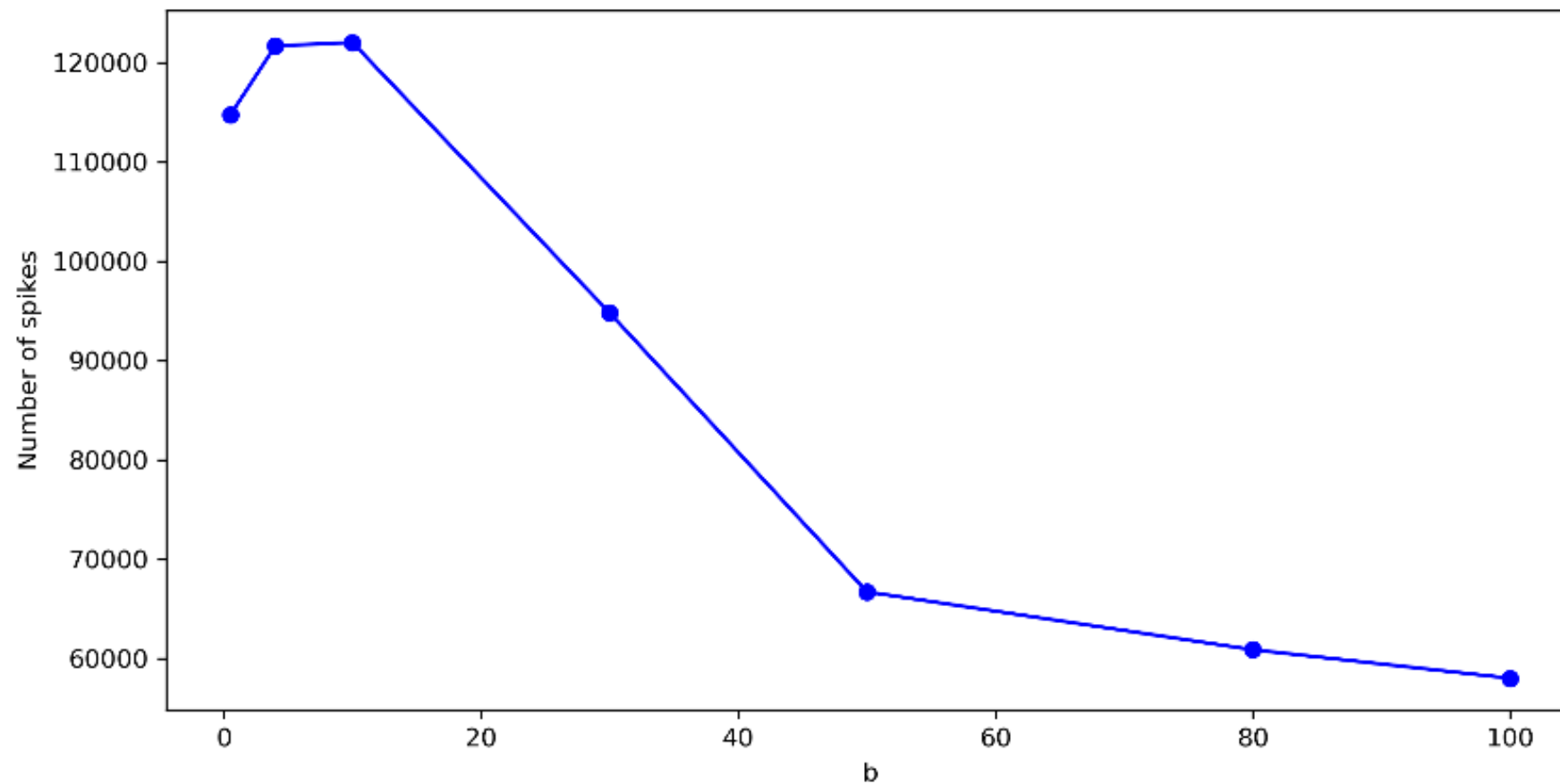


Download website: <https://github.com/cvdfoundation/mnist>

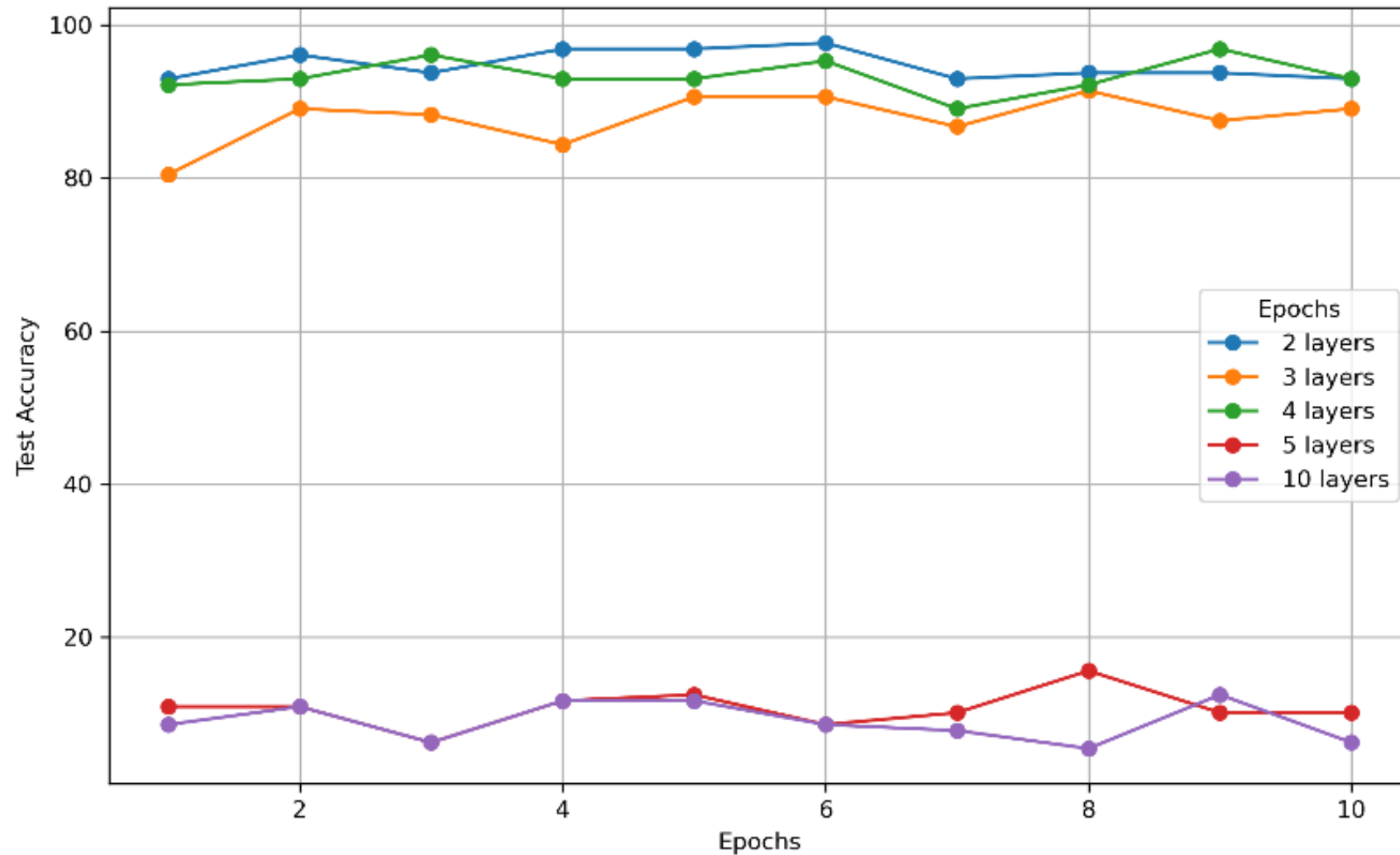
Dspike - accuracy vs b (2 layers)



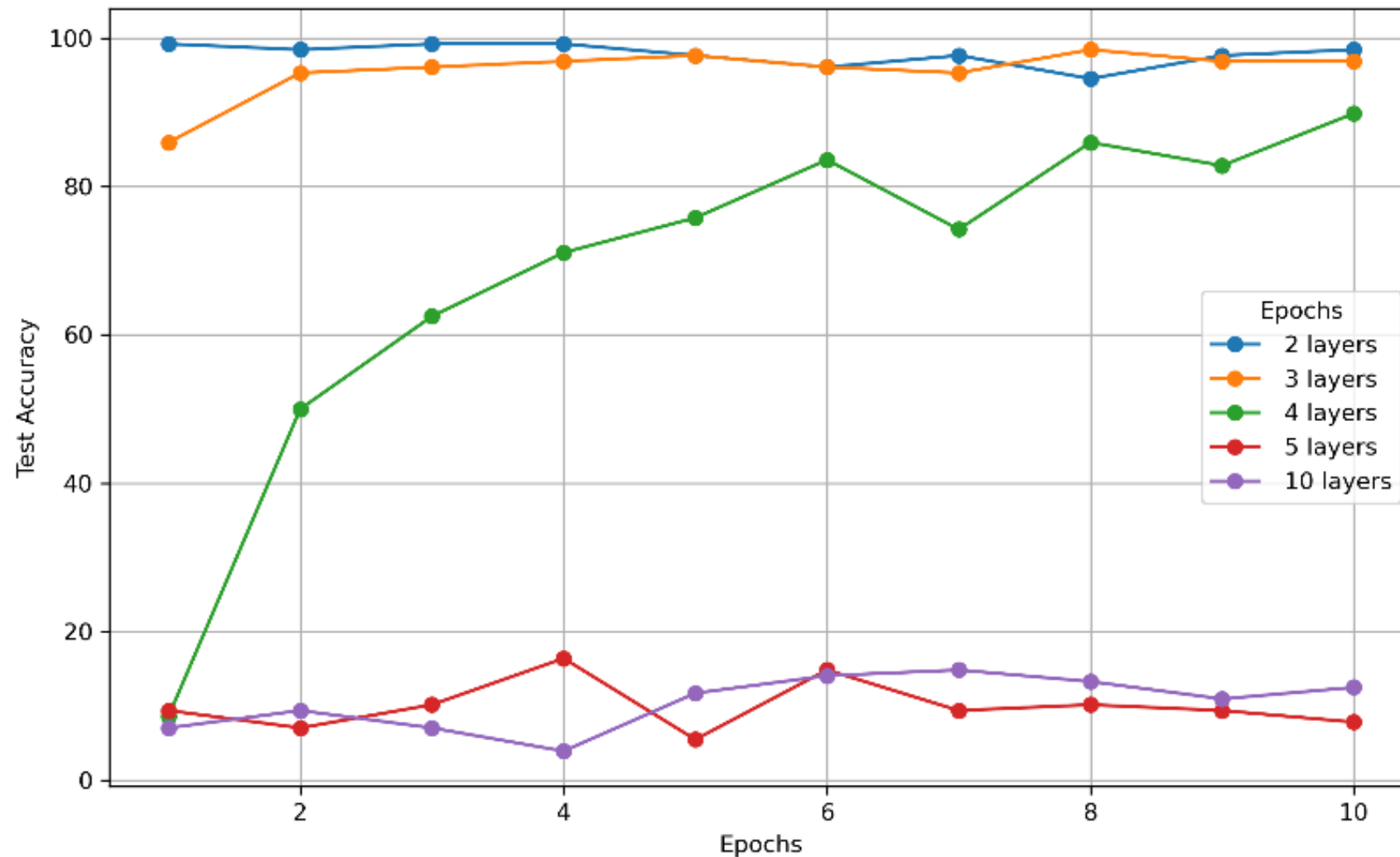
Dspike - no. of spikes vs b



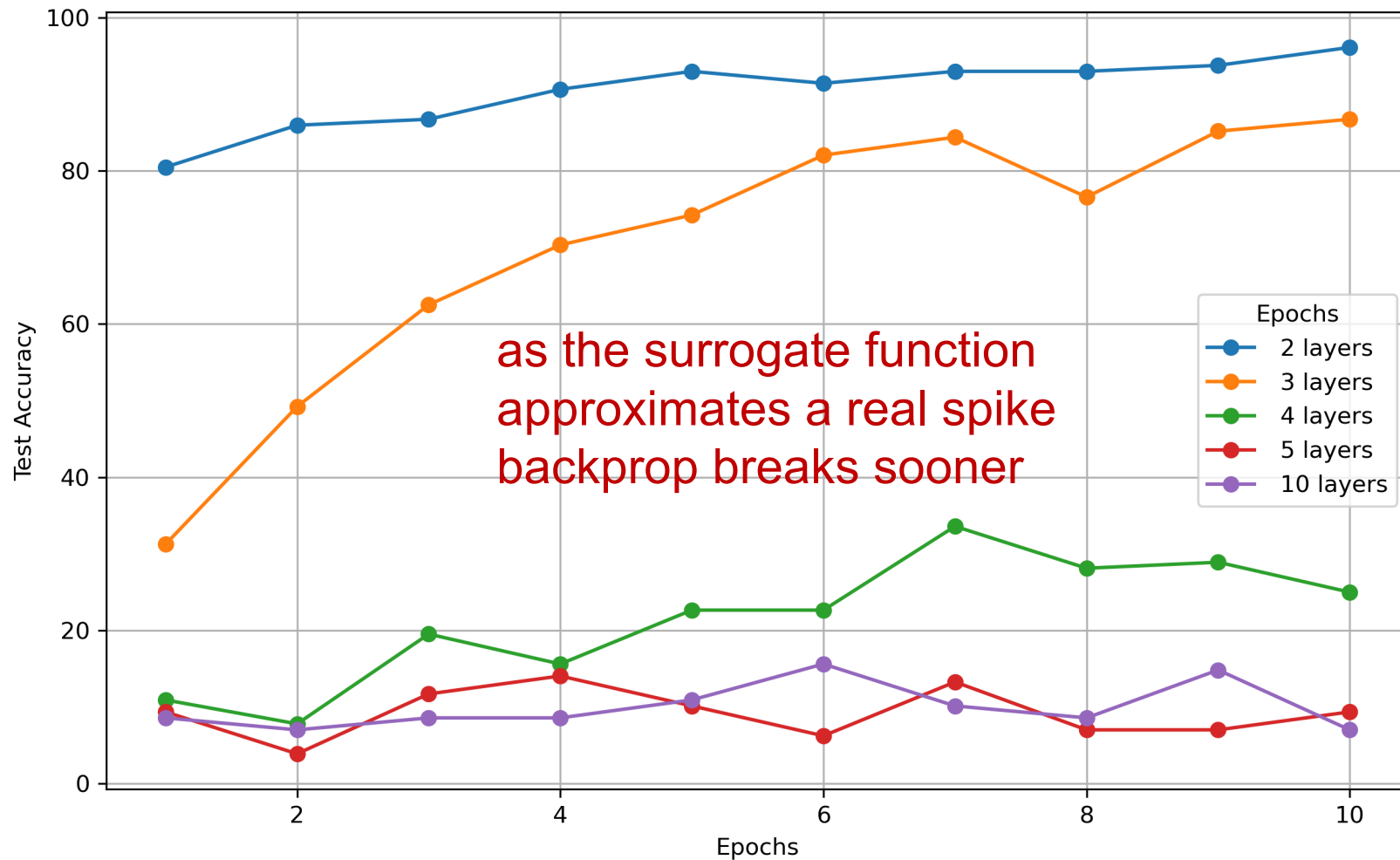
Test accuracy vs training time ($b=0.5$)



Test accuracy vs training time (b=100)



Test accuracy vs training time (b=1000)



Residual connections: a step forward

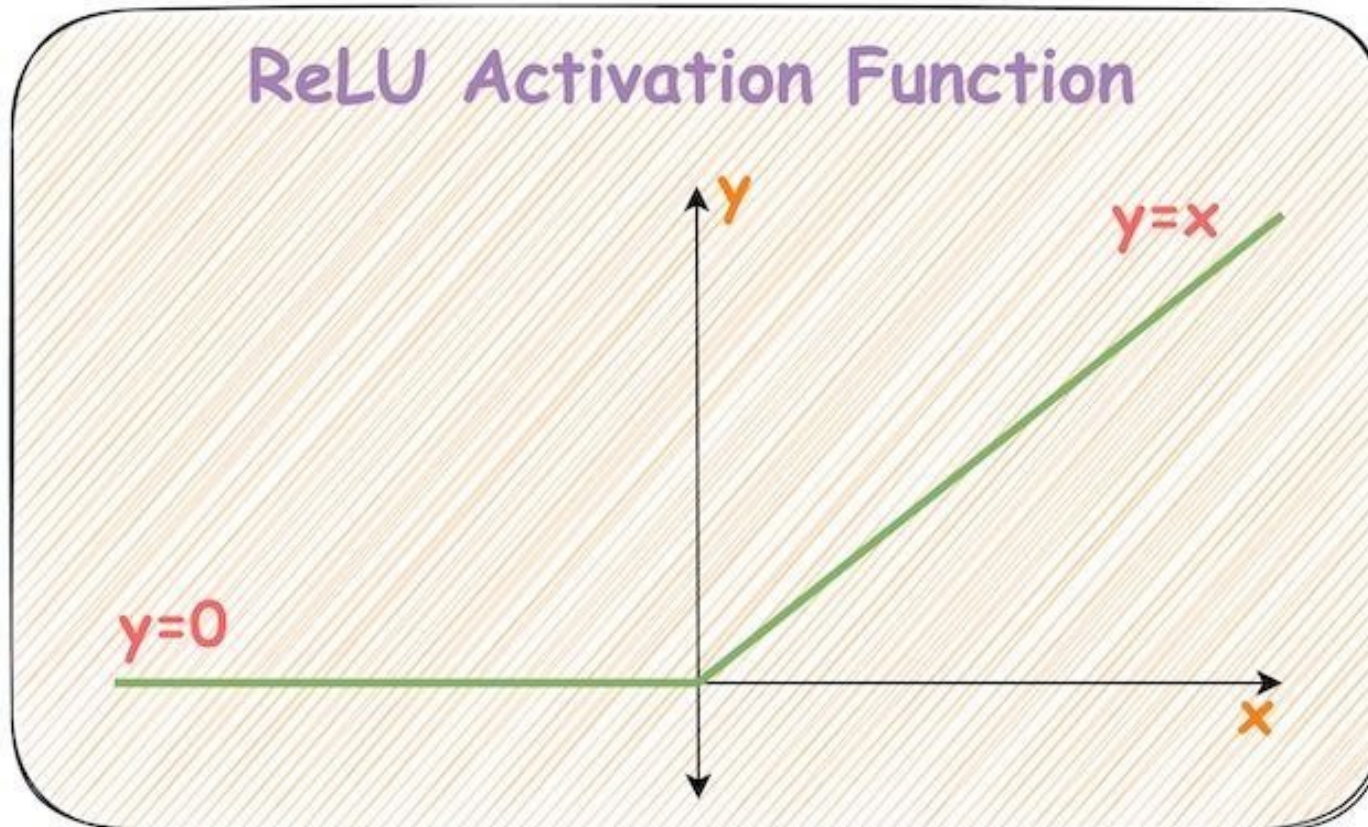
[He2016] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun, “Deep Residual Learning for Image Recognition,” IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, US, 2016. [Best Paper Award]

[He2016-b] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun, “Identity Mappings in Deep Residual Networks,” European Conference on Computer Vision (ECCV), Amsterdam, The Netherlands, October 2016.

Deep ResNet with 1k layers

<https://github.com/KaimingHe/resnet-1k-layers>

Rectified Linear Unit (ReLU)



$$y = \max(x, 0)$$

ResNets idea

Key idea

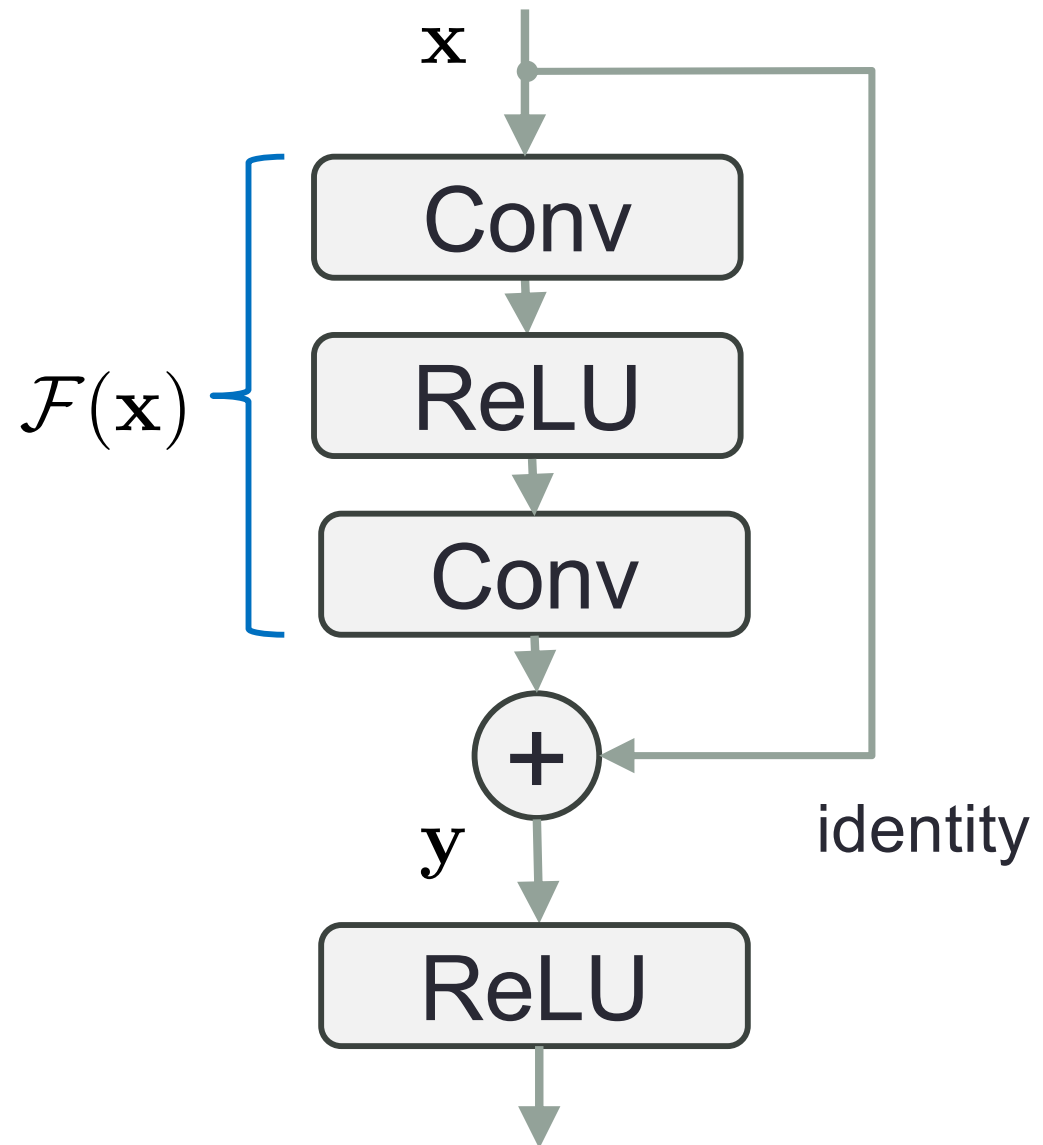
$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$$

is the desired output
feature vector

$\mathbf{x}$ is the input vector,
which is known

the network learns **the
residual**

$$\mathcal{F}(\mathbf{x}) = \mathbf{y} - \mathbf{x}$$



Residual units

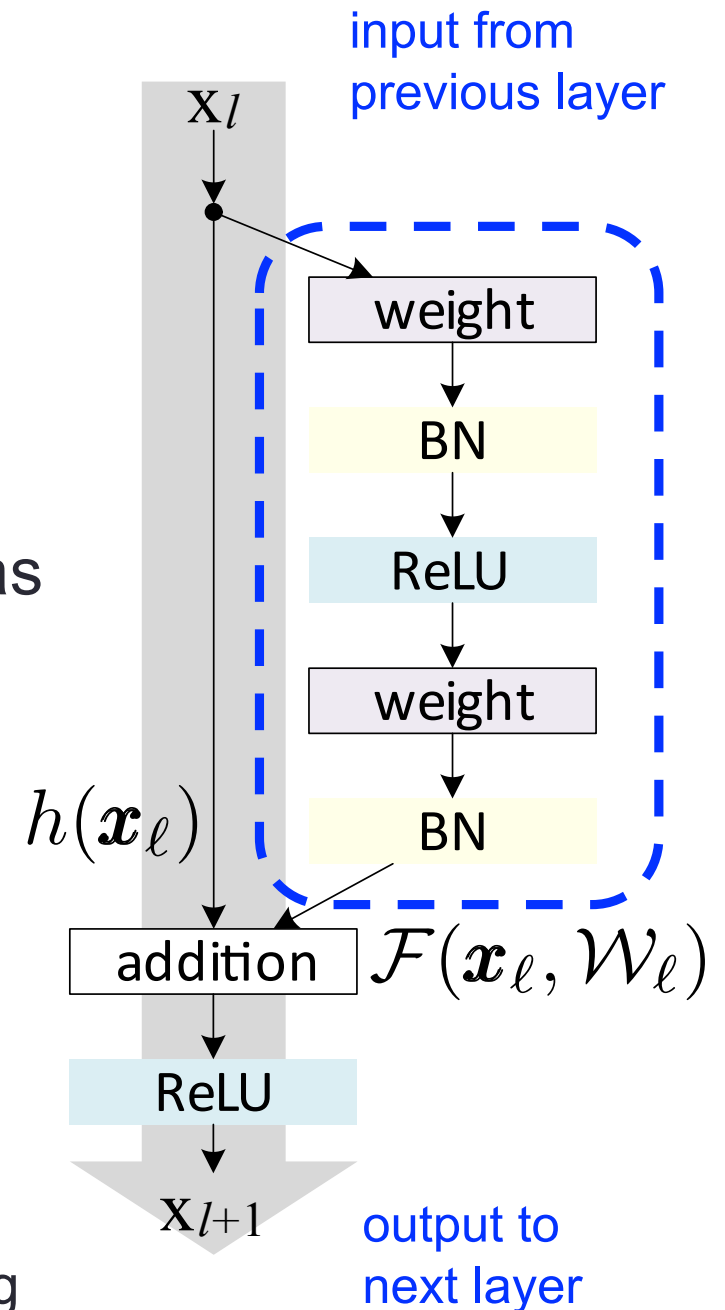
Formally

- a **new way** of impl. network layers
- stacking so **called residual units**
- each **residual unit** can be expressed as

$$\mathbf{y}_\ell = h(\mathbf{x}_\ell) + \mathcal{F}(\mathbf{x}_\ell, \mathcal{W}_\ell)$$

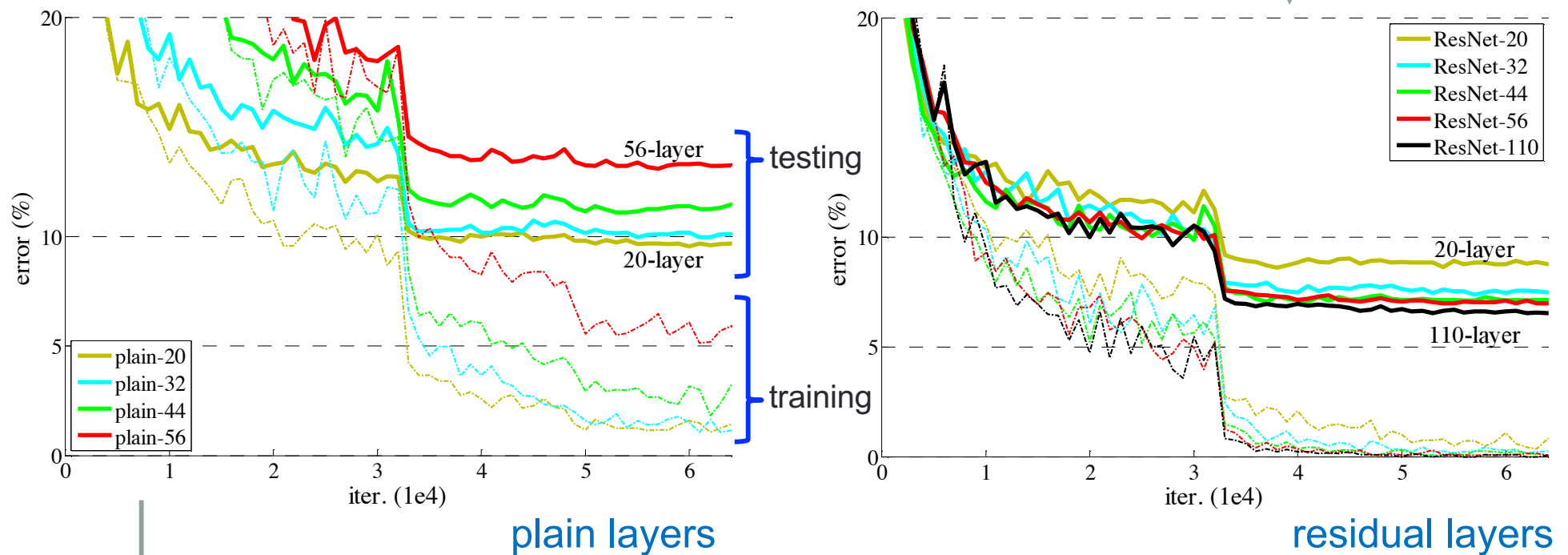
$$\mathbf{x}_{\ell+1} = f(\mathbf{y}_\ell)$$

- where
 - **$h()$ is an identity mapping** - key choice!
 - implemented using a *skip connection*
 - **$f()$ is a ReLU non-linearity**
 - **$\mathcal{F}(\mathbf{x}_\ell, \mathcal{W}_\ell)$** (non-linear) residual mapping



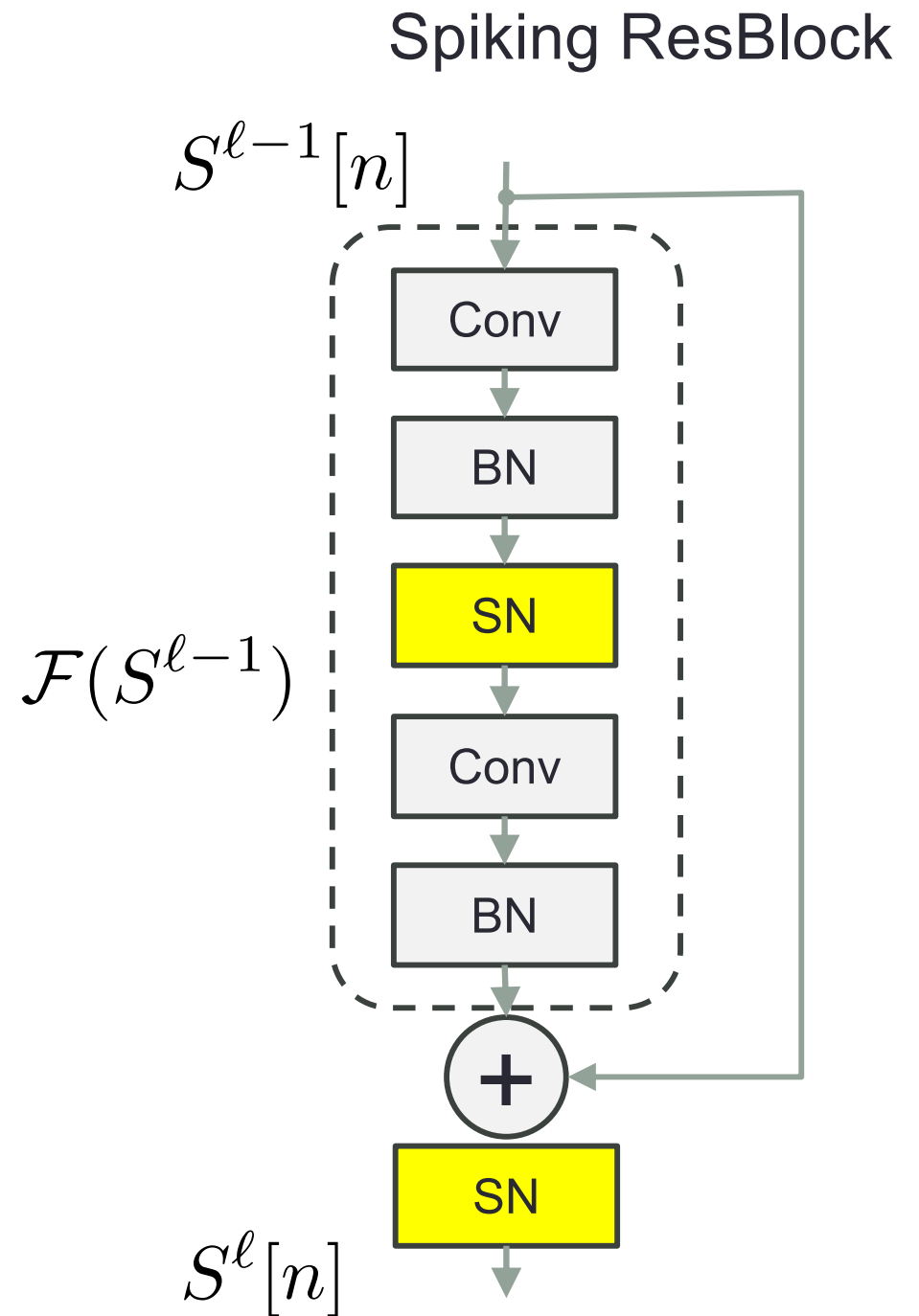
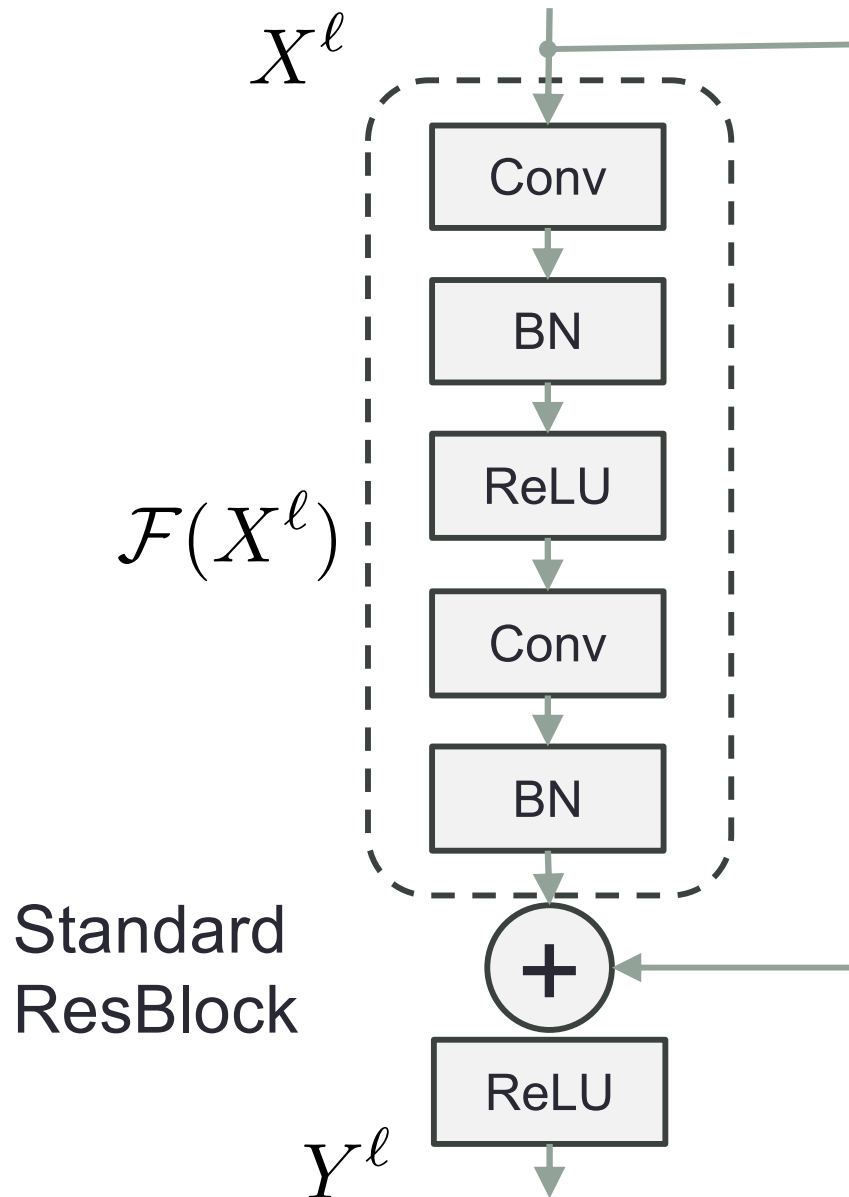
Results on CIFAR-10

ResNets can get deeper and still improve their performance



performance degrades by increasing no of layers

ResNets



Analysis: standard ResBlock

- The input/output relationship of a residual block is

$$Y^\ell = \text{ReLU}(\mathcal{F}(X^\ell) + X^\ell)$$

- When $\mathcal{F}(X^\ell)[n] = 0$ the output is

$$Y^\ell = \text{ReLU}(X^\ell)$$

- Where in most cases (read “always”) X^ℓ is the last activation of the previous residual block, which corresponds to the output of a ReLU
- Hence, we have

$$Y^\ell = \text{ReLU}(X^\ell) = X^\ell \quad \text{identity}$$

Analysis: spiking ResBlock

- With the spiking residual block it holds

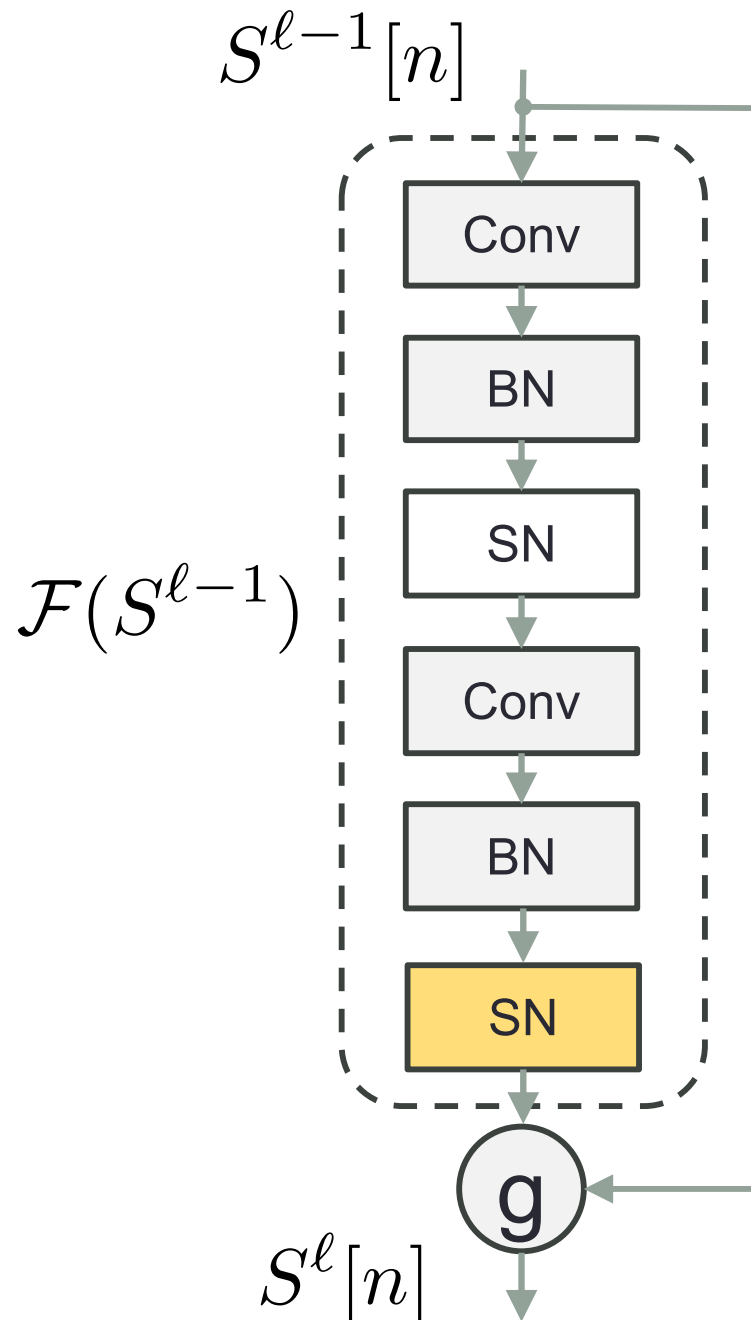
$$S^\ell[n] = SN(\mathcal{F}(S^{\ell-1})[n] + S^{\ell-1}[n])$$

- When $\mathcal{F}(S^{\ell-1})[n] = 0$ the output is

$$S^\ell[n] = SN(\mathcal{F}(S^{\ell-1}[n])) \neq S^{\ell-1}[n]$$

NOT an identity (due to Heaviside)

- **To obtain an identity** (what we need) the last spiking neuron (SN block) in the residual block needs
 - to fire a spike after receiving a spike and
 - remain silent when no spikes are received at time n



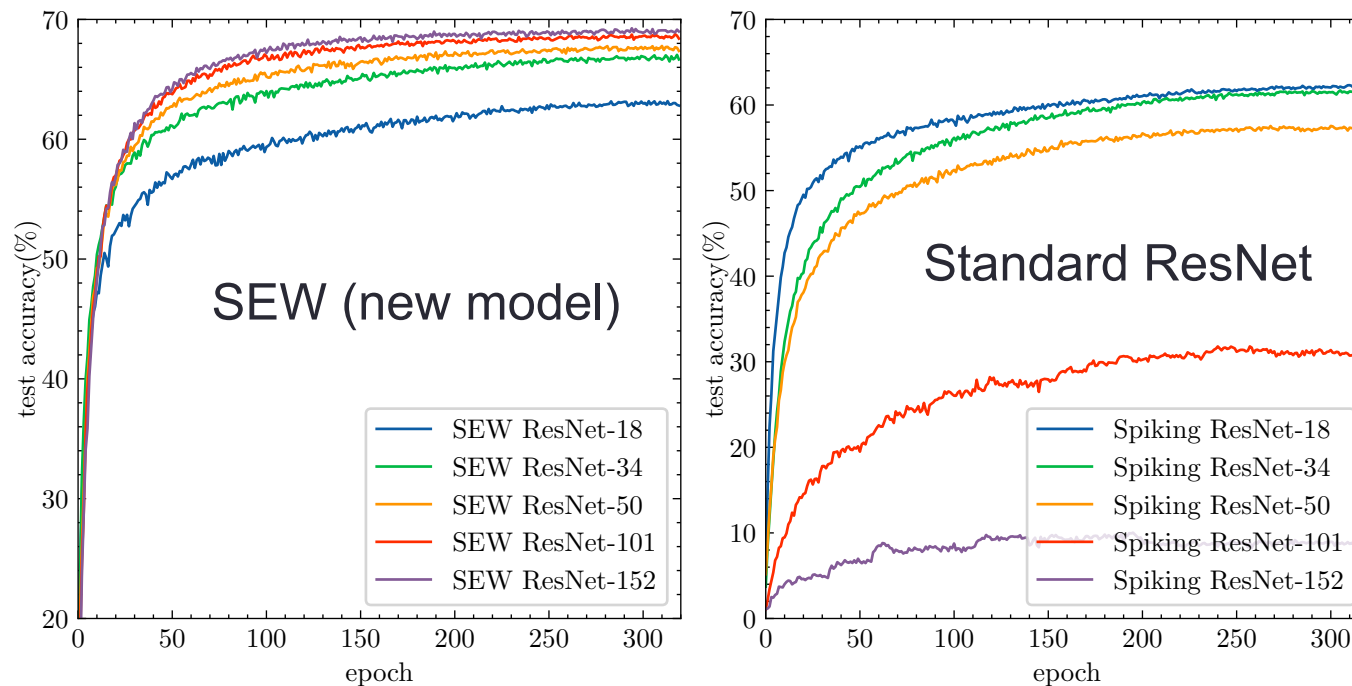
Solution

Moving the SN block before the sum

This new design, with the following g functions behaves as an identity when $A[n] = 0$

$$g(A[n], S[n]) = \begin{cases} A[n] + S[n] \\ (1 - A[n])S[n] \end{cases}$$

ImageNet – test accuracy



Network	SEW ResNet (ADD)		Spiking ResNet	
	Acc@1(%)	Acc@5(%)	Acc@1(%)	Acc@5(%)
ResNet-18	63.18	84.53	62.32	84.05
ResNet-34	67.04	87.25	61.86	83.69
ResNet-50	67.78	87.52	57.66	80.43
ResNet-101	68.76	88.25	31.79	54.91
ResNet-152	69.26	88.57	10.03	23.57

Future work

- Supervised
 - Integration with attention mechanisms
 - Alternative training methods (ADMM-based?)
- Unsupervised learning
 - Still underperforming
- Mixed unsupervised+supervised
 - Spike-time dependent plasticity (STDP)
 - Reward modulated STDP
 - Fluid state machines
- Federated learning with distributed SNN agents
 - Still open

Bibliography (1)

[SNN-Book-2014] Wulfram Gerstner, “Neuronal Dynamics: from Single Neurons to Networks and Modes of Cognition,” *Cambridge University Press*, Sept. 2014.

[SPM-2019] Emre O. Neftci, Hesham Mostafa, and Friedemann Zenke, “Surrogate Gradient Learning in Spiking Neural Networks: bringing the power of gradient-based optimization to spiking neural networks,” *IEEE Signal Processing Magazine*, 2019.

[ProcIEEE-2023] Jason K. Eshraghian, Max Ward, Emre O. Neftci, Xinxin Wang, Gregor Lenz, Girish Dwivedi, Mohammed Bennamoun, Doo Seok Jeong, Wei D. Lu, “Training Spiking Neural Networks Using Lessons From Deep Learning,” *Proceedings of the IEEE*, 2023.

[SNNtorch] Jason K. Eshraghian, “Tutorial 5: Training Spiking Neural Networks with SNN Torch,” *online*, 2021.

https://snntorch.readthedocs.io/en/latest/tutorials/tutorial_5.html

Bibliography (2)

[He2016] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun, “Deep Residual Learning for Image Recognition,” IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, US, 2016. *Best Paper Award*

[He2016-b] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun, “Identity Mappings in Deep Residual Networks,” European Conference on Computer Vision (ECCV), Amsterdam, The Netherlands, October 2016.

[NeurIPS-2021] W. Fang, Z. Yu, Y. Chen, T. Huang, T. Masquelier, Y. Tian, “Deep Residual Learning in Spiking Neural Networks,” NeurIPS 2021.

SPIKING NEURAL NETWORKS

Michele Rossi

michele.rossi@unipd.it

Dept. of Information Engineering

Dept. of Mathematics



**UNIVERSITÀ
DEGLI STUDI
DI PADOVA**